# Содержание

**1. Руководство пользователя** 7
1.1. О продукте 7
1.2. Подготовка к работе 7
1.3. Включение расширения в проект автоматизации 8
1.4. Примеры использования компонентов расширения 10
1.4.1. Выбор цвета на панели инструментов из палитры или из меню 10
1.4.2. Подключение к серверу для просмотра дерева сигналов 12
1.4.3. Изменение темы оформления 16

**2. Справочное руководство** 18
2.1. Типы 18
2.1.1. Тема оформления 18
2.1.1.1. Свойства 19
2.1.1.2. Функции 21
2.1.2. Конвертер значений 22
2.1.2.1. Функции 23
2.1.3. Утилиты для работы со строками 28
2.1.3.1. Функции 28
2.1.4. Утилиты для работы с окном 29
2.1.4.1. Свойства 29
2.1.4.2. Функции 29
2.1.5. Утилиты для работы с файлами 31
2.1.5.1. Свойства 31
2.1.5.2. Функции 32
2.1.6. Информация о продукте 32
2.1.6.1. Свойства 32
2.1.6.2. Функции 33
2.1.7. Буфер аргументов события 34
2.1.7.1. Свойства 36
2.1.7.2. Функции 36
2.1.8. Обработчик события 36
2.1.8.1. Свойства 37
2.1.8.2. Функции 38
2.1.8.3. События 38
2.1.9. Пользовательская команда 38
2.1.9.1. Свойства 41
2.1.9.2. События 42
2.1.9.3. Функции 42
2.1.10. Текстовые шаблоны 42
2.1.10.1. Свойства 43
2.1.10.2. Функции 43
2.1.11. Редакторы 45
2.1.11.1. Редактор целого числа 45
Свойства 46
Функции 47
2.1.11.2. Редактор даты 47
Свойства 48
Функции 49
2.1.11.3. Редактор времени 50
Свойства 50
Функции 52
2.1.11.4. Редактор интервала времени 52
Свойства 52

2.1.11.5. Горизонтальный сплиттер ... 53
Свойства ... 54
2.1.11.6. Вертикальный сплиттер ... 55
Свойства ... 56
2.1.11.7. Рамка с ошибкой ... 57
Свойства ... 58
2.1.11.8. Редактор даты и времени ... 58
Свойства ... 59
Функции ... 60
2.1.12. Меню ... 60
2.1.12.1. Элемент меню ... 60
Свойства ... 61
2.1.12.2. Разделитель элементов меню ... 62
2.1.12.3. Результат выбора меню ... 62
Свойства ... 62
2.1.13. Панель инструментов ... 63
2.1.13.1. Кнопка панели инструментов ... 64
Свойства ... 64
2.1.13.2. Кнопка выпадающего списка ... 65
Свойства ... 66
2.1.13.3. Разделитель панели инструментов ... 67
Свойства ... 67
2.1.13.4. Панель поиска ... 67
Свойства ... 67
Функции ... 68
2.1.13.5. Тема оформления панели инструментов ... 68
Свойства ... 68
2.1.14. Строка статуса ... 71
2.1.14.1. Кнопка авторизации ... 71
Свойства ... 71
2.1.14.2. Кнопка-индикатор журнала сообщений ... 71
Свойства ... 72
2.1.14.3. Индикатор подключения к серверу ... 72
Свойства ... 72
2.1.14.4. Кнопка смены раскладки клавиатуры ... 73
2.1.14.5. Кнопка вызова экранной клавиатуры ... 73
Свойства ... 73
2.1.14.6. Параметры вызова экранной клавиатуры ... 73
Свойства ... 74
2.1.15. Диалоги ... 75
2.1.15.1. Нажата клавиша Caps Lock ... 75
2.1.15.2. Результат выбора папки ... 75
Свойства ... 75
2.1.15.3. Результат выбора цвета ... 75
Свойства ... 75
2.1.15.4. Результат выбора параметров подключения ... 76
Свойства ... 76
2.1.15.5. Результат выбора даты и времени ... 77
Свойства ... 77
2.1.15.6. Результат ввода значения ... 77
Свойства ... 77
2.1.15.7. Результат редактирования колонтитула ... 78
Свойства ... 79
2.1.15.8. Результат выбора файла ... 79
Свойства ... 79
2.1.15.9. Результат диалога авторизации ... 79

Свойства 80
2.1.15.10. Список файлов 80
Свойства 80
2.1.16. Дерево сигналов 83
2.1.16.1. Свойства 83
2.1.16.2. Функции 86
2.1.17. Журнал сообщений 88
2.1.17.1. Свойства 88
2.1.17.2. Функции 90
2.1.17.3. Команды 93
2.1.18. Редактируемая таблица 93
2.1.18.1. Свойства 96
2.1.18.2. Функции 102
2.1.19. Редактор ячейки таблицы 105
2.1.19.1. Свойства 106
2.2. Экранные формы 107
2.2.1. Popup 107
2.2.2. Меню 107
2.2.2.1. Свойства 107
2.2.3. Календарь 110
2.2.3.1. Свойства 111
2.2.4. Сообщение 111
2.2.4.1. Свойства 113
2.2.5. Диалог ввода значения 115
2.2.5.1. Свойства 115
2.2.6. Диалог выбора цвета 116
2.2.6.1. Свойства 117
2.2.7. Диалог подключения к серверу 118
2.2.7.1. Свойства 118
2.2.8. Редактор колонтитула 120
2.2.8.1. Свойства 121
2.2.9. Файлы 122
2.2.9.1. Диалог открытия файла 122
Свойства 122
2.2.9.2. Диалог сохранения файла 127
Свойства 127
2.2.9.3. Диалог выбора папки 132
Свойства 132
2.2.10. Безопасность 133
2.2.10.1. Диалог авторизации пользователя 133
Свойства 135
2.2.10.2. Диалог смены пароля 136
Свойства 136
2.2.10.3. Диалог ввода учетных данных 137
Свойства 137
2.3. Общие свойства компонентов библиотеки 137
2.3.1. Тема оформления 137

**История изменений 139**
2.4 139
Изменения документации 140
Редакция 1 140
2.3 140
Изменения документации 140
Редакция 1 140
2.2 141

Изменения документации ..... 141
Редакция 1 ..... 141
Редакция 2 ..... 142
2.1 ..... 142
2.1.1 ..... 142
2.1.2 ..... 143
2.1.3 ..... 143
Изменения документации ..... 143
Редакция 1 ..... 143
Редакция 2 ..... 144
Редакция 3 ..... 144
Редакция 4 ..... 144
Редакция 5 ..... 144
2.0 ..... 144
2.0.1 ..... 145
2.0.2 ..... 145
Изменения документации ..... 146
Редакция 1 ..... 146
Редакция 2 ..... 146
Редакция 3 ..... 147
1.7 ..... 147
1.7.1 ..... 147
1.7.2 ..... 148
1.7.3 ..... 148
Изменения документации ..... 148
Редакция 1 ..... 148
Редакция 2 ..... 148
1.6 ..... 149
1.6.1 ..... 149
Изменения документации ..... 149
Редакция 1 ..... 149
Редакция 2 ..... 150
1.5 ..... 150
1.5.2 ..... 150
Изменения документации ..... 151
Редакция 1 ..... 151
Редакция 2 ..... 151
1.4 ..... 151
1.4.1 ..... 152
Изменения документации ..... 152
Редакция 1 ..... 152
Редакция 2 ..... 153
1.3 ..... 153
1.3.1 ..... 153
1.3.2 ..... 153
1.3.3 ..... 154
Изменения документации ..... 154
Редакция 1 ..... 154
Редакция 2 ..... 154
Редакция 3 ..... 155
1.2 ..... 155
Изменения документации ..... 155
Редакция 1 ..... 155

# 1. Руководство пользователя

## 1.1. О продукте

Alpha.HMI.CommonLib - это расширение среды разработки и исполнения Alpha.HMI.

Расширение представляет собой библиотеку компонентов, которые можно использовать в своих проектах Alpha.HMI. Среди них - различные диалоговые окна, кнопки, файловые менеджеры, конвертеры значений и многое другое. Внешний вид и функциональность этих компонентов можно настраивать под свои проекты с помощью свойств, функций и событий. Подробнее состав библиотеки, а также назначение, свойства, функции и события каждого компонента описаны в Справочном руководстве.

Расширение также используется для работы некоторых компонентов Альфа платформы. Поэтому его часто необходимо устанавливать и встраивать в проекты автоматизации, разработанные в Alpha.HMI, наряду с другими расширениями.

## 1.2. Подготовка к работе

### Требования к окружению

Для использования средств Alpha.HMI.CommonLib должны быть установлены:

- среда разработки и исполнения Alpha.HMI;
- подсистема безопасности Alpha.Security;
- расширения Alpha.HMI.Tables и Alpha.HMI.Security среды Alpha.HMI.

Если работаете с веб-версией проекта автоматизации, установите дополнительно:

- Alpha.HMI.WebViewer - для использования проектов Alpha.HMI в веб-интерфейсе;
- Alpha.HMI.Security.WebViewer - для работы компонентов Alpha.HMI.Security в веб-интерфейсе;
- Alpha.HMI.Tables.WebViewer - для работы компонентов Alpha.HMI.Tables в веб-интерфейсе.

### Установка и удаление на ОС Windows

Для установки, удаления или восстановления Alpha.HMI.CommonLib используйте мастер. Запустите установочный файл `alpha.hmi.commonlib-<lng>-<version>.<arch>.msi`.

> ПРИМЕЧАНИЕ
> В названии файла `<lng>` - это язык продукта, `<version>` - версия продукта, `<arch>` - целевая процессорная архитектура.

Следуйте инструкциям мастера.

Расширение устанавливается в папку `C:\Program Files\Automiq\Alpha.HMI.Extensions\CommonLib` или `C:\Program Files (x86)\Automiq\Alpha.HMI.Extensions\CommonLib`.

### Установка и удаление на ОС Linux

Для установки Alpha.HMI.CommonLib вызовите пакет с командой на установку:

- rpm-пакет:

```
sudo rpm -i alpha.hmi.commonlib-<lng>-<version>.rpm
```

- deb-пакет:

```
sudo dpkg -i alpha.hmi.commonlib-<lng>-<version>.deb
```

ПРИМЕЧАНИЕ

В названии файла `<lng>` - это язык продукта, а `<version>` - версия продукта.

Расширение устанавливается в папку `/opt/Automiq/Alpha.HMI.Extensions/CommonLib`.

Для удаления Alpha.HMI.CommonLib вызовите пакет с командой на удаление:

- rpm-пакет:

```
sudo rpm -e alpha.hmi.commonlib
```

- deb-пакет:

```
sudo dpkg -r alpha.hmi.commonlib
```

## 1.3. Включение расширения в проект автоматизации

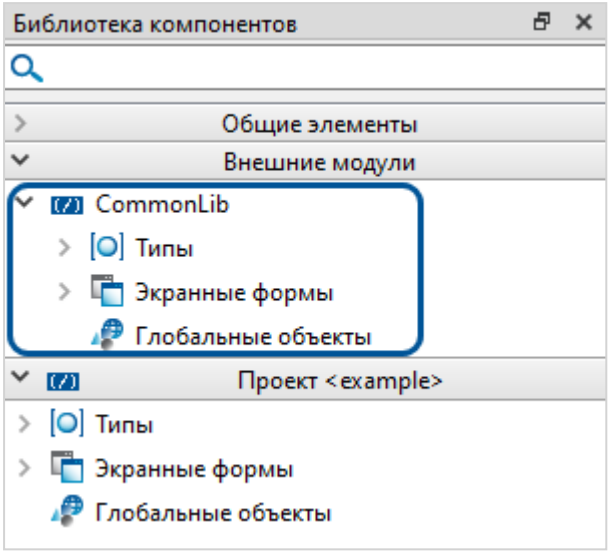
Расширение Alpha.HMI.CommonLib выполнено в виде проекта Alpha.HMI. Чтобы использовать компоненты расширения, необходимо подключить проект расширения в свой проект Alpha.HMI в виде внешнего модуля.

1. Чтобы подключать проекты расширений в виде внешних модулей, создайте в папке своего проекта папку с именем `externals`. Если к проекту уже подключены другие модули, папка `externals` может уже существовать.

| Имя | Дата изменения | Тип | Размер |
|---|---|---|---|
| externals | 06.12.2022 10:51 | Папка с файлами | |
| objects | 17.11.2022 16:49 | Папка с файлами | |
| resources | 02.11.2022 11:10 | Папка с файлами | |
| example.hmi | 06.12.2022 11:18 | Alpha.HMI project | 1 КБ |

2. В созданную папку скопируйте папку расширения `CommonLib`, расположенную в:
   - `C:\Program Files\Automiq\Alpha.HMI.Extensions` или `C:\Program Files (x86)\Automiq\Alpha.HMI.Extensions` (ОС Windows);
   - `/opt/Automiq/Alpha.HMI.Extensions` (ОС Linux).

3. Откройте свой проект в Дизайнер Alpha.HMI. В меню **Проект** на панели инструментов выберите пункт **Обновить ссылки на внешние юниты**. В результате в библиотеке компонентов появится юнит внешнего модуля **CommonLib**.




Чтобы приступить к работе, перетяните компонент на форму или вызовите из обработчика какого-либо события. В проект можно добавить столько экземпляров одного типа, сколько необходимо. Разные экземпляры одного типа можно использовать под разные цели.

### Обновление расширения

Если Alpha.HMI.CommonLib уже встроен в проект в виде внешнего модуля, то при установке новой версии необходимо обновить файлы модуля в папке `externals` проекта. Для этого:

1. Установите нужную версию Alpha.HMI.CommonLib.
2. Замените существующую папку `CommonLib` на папку `CommonLib`, расположенную в папке установки расширения:
   - `C:\Program Files\Automiq\Alpha.HMI.Extensions` или `C:\Program Files (x86)\Automiq\Alpha.HMI.Extensions` (ОС Windows);
   - `/opt/Automiq/Alpha.HMI.Extensions` (ОС Linux).
3. Откройте свой проект в Дизайнер Alpha.HMI. В меню **Проект** на панели инструментов выберите **Обновить ссылки на внешние юниты**. В результате внешние модули будут обновлены. Экземпляры компонентов Alpha.HMI.CommonLib, уже добавленные в проект, обновятся автоматически.

# 1.4. Примеры использования компонентов расширения

## 1.4.1. Выбор цвета на панели инструментов из палитры или из меню

ПРИМЕЧАНИЕ

В разделе продемонстрировано применение компонентов 2.1.13.1. Кнопка панели инструментов (стр. 64), 2.1.13.2. Кнопка выпадающего списка (стр. 65), 2.2.6. Диалог выбора цвета (стр. 116) и 2.2.2. Меню (стр. 107). Для этого в Alpha.HMI реализован проект-пример, который можно скачать для ознакомления.

Предположим, необходимо создать панель инструментов, предназначенную для выбора цвета из палитры или из выпадающего меню. Решим эту задачу, использовав комбинацию компонентов 2.1.13.1. Кнопка панели инструментов (стр. 64), 2.1.13.2. Кнопка выпадающего списка (стр. 65), 2.2.6. Диалог выбора цвета (стр. 116) и 2.2.2. Меню (стр. 107):

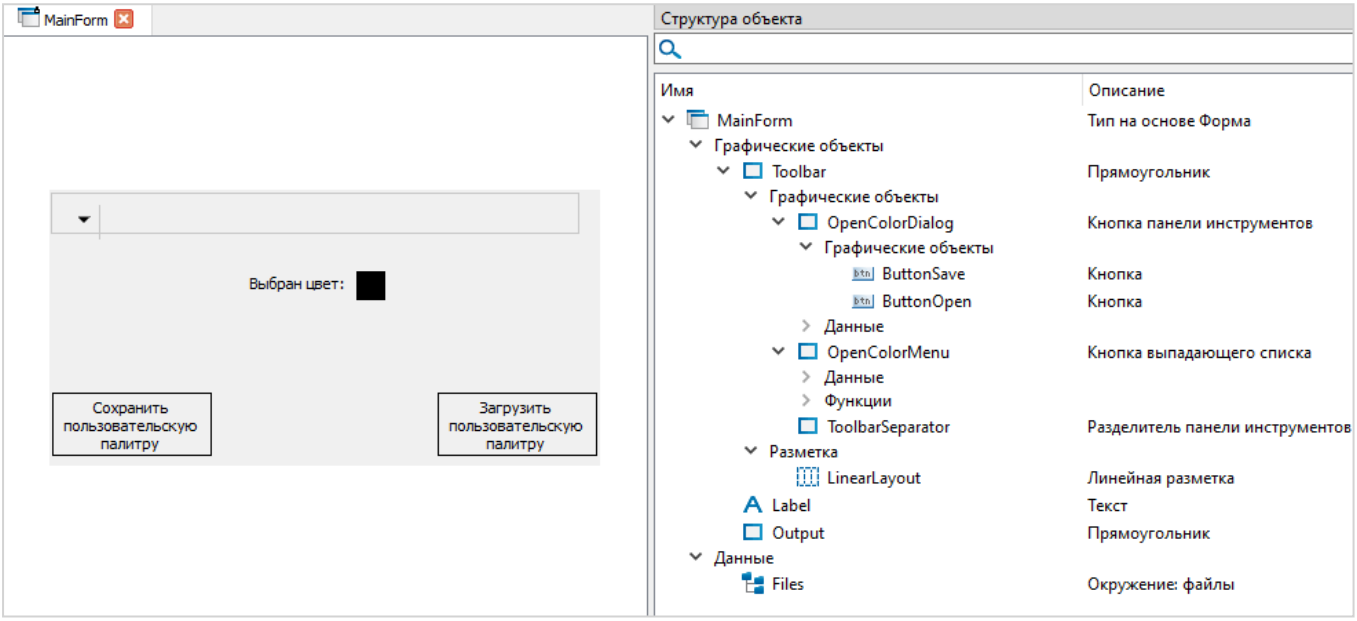
- По нажатию на кнопку типа 2.1.13.1. Кнопка панели инструментов (стр. 64) открывается палитра 2.2.6. Диалог выбора цвета (стр. 116). Пусть при этом необходимо иметь возможность использовать пользовательскую палитру цветов.
- По нажатию на кнопку типа 2.1.13.2. Кнопка выпадающего списка (стр. 65) открывается 2.2.2. Меню (стр. 107), где можно выбрать цвет из предложенных.

Решение реализовано в проекте-примере. Скачайте проект-пример и откройте в Дизайнер Alpha.HMI для наглядности.

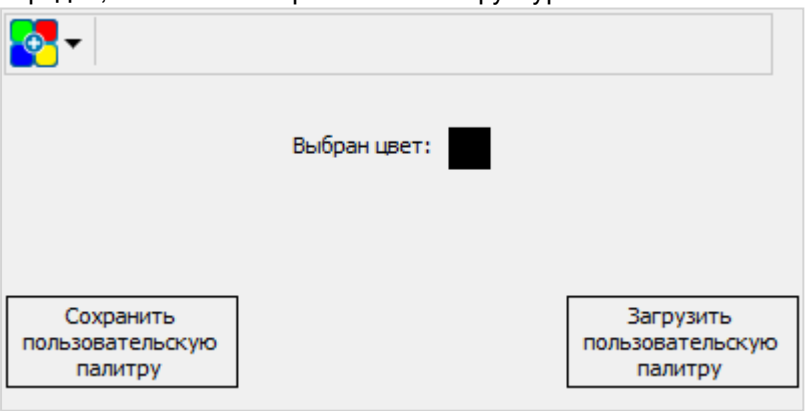
К проекту подключен внешний модуль **`CommonLib`**. Проект содержит единственную форму *`MainForm`*. На ней расположены панель инструментов *`Toolbar`* и прямоугольник *`Output`*, окрашивающийся в цвет, выбранный с помощью панели инструментов. К панели инструментов *`Toolbar`* применен инструмент линейной разметки. Также на форме расположен вспомогательный экземпляр *`Files`* компонента **`Окружение: файлы`**, служащий для сохранения и открытия файлов.

На панели инструментов размещены:

- кнопка *OpenColorDialog* - кнопка типа 2.1.13.1. Кнопка панели инструментов (стр. 64), по щелчку ЛКМ на которую открывается диалоговое окно для выбора цвета из палитры 2.2.6. Диалог выбора цвета (стр. 116);
- кнопка *OpenColorMenu* - кнопка типа 2.1.13.2. Кнопка выпадающего списка (стр. 65), по щелчку ЛКМ на которую открывается выпадающее меню для выбора цвета из списка;
- кнопки *ButtonSave* и *ButtonOpen*, предназначенные для сохранения и загрузки пользовательской палитры цветов, выбранной в окне палитры 2.2.6. Диалог выбора цвета (стр. 116);
- разделитель элементов типа 2.1.13.3. Разделитель панели инструментов (стр. 67).




Запустите проект-пример в рантайм. Обратите внимание, что элементы панели расположены в таком порядке, в каком они перечислены в структуре объекта.




Щелкните ЛКМ по кнопке выбора цвета из палитры (*OpenColorDialog*, ). Активируется событие

MouseClick кнопки, открывающее 2.2.6. Диалог выбора цвета (стр. 116) в новом окне. Ознакомьтесь с обработчиком события.

- Свойство Result компонента 2.2.6. Диалог выбора цвета (стр. 116) записывает результат выбора цвета в экземпляр 2.1.15.3. Результат выбора цвета (стр. 75), расположенный в структуре объекта внутри *OpenColorDialog*.
- Свойство okAction компонента 2.2.6. Диалог выбора цвета (стр. 116) ссылается на команду *onClick*, расположенную в структуре объекта внутри *OpenColorDialog*. Команда выполняется в момент нажатия кнопки **OK** в окне 2.2.6. Диалог выбора цвета (стр. 116) и окрашивает прямоугольник *Output* в цвет, записанный в *ColorDialogResult*.

Выберите цвет из палитры и нажмите OK. Прямоугольник окрасится в выбранный цвет.

Теперь щелкните ЛКМ по кнопке, открывающей выпадающее меню (*OpenColorMenu*, ). Активируется событие MouseClick кнопки, открывающее 2.2.6. Диалог выбора цвета (стр. 116) в новом окне. Ознакомьтесь с обработчиком события.

- Свойство Items компонента 2.2.2. Меню (стр. 107) ссылается на функцию GetMenuItems(), расположенную в структуре объекта внутри *OpenColorMenu*. Функция описывает пункты меню в виде JSON массива. Ознакомьтесь с описанием функции.
- Свойство pResult компонента 2.2.2. Меню (стр. 107) записывает id выбранного пункта меню в экземпляр 2.1.12.3. Результат выбора меню (стр. 62), расположенный в структуре объекта внутри *OpenColorMenu*.
- Свойство pAction компонента 2.2.2. Меню (стр. 107) ссылается на команду *onClick*, расположенную в структуре объекта внутри *OpenColorMenu*. Команда выполняется в момент выбора пункта меню и окрашивает прямоугольник *Output* в цвет, соответствующий id выбранного пункта меню.

Выберите цвет из меню. Прямоугольник окрасится в выбранный цвет.

Сохраним пользовательскую палитру в файл.

1. Откройте окно выбора цвета вновь и добавьте несколько цветов к пользовательской палитре.
2. Закройте окно выбора цвета и нажмите **Сохранить пользовательскую палитру**. Активируется событие ButtonPressed кнопки. Ознакомьтесь с обработчиком события. Экземпляр *Files* компонента **Окружение: файлы** формирует путь к файлу `colors.json`, создаваемом в папке проекта `output`. Затем туда записывается пользовательская палитра в виде JSON-строки из свойства CustomColors компонента 2.2.6. Диалог выбора цвета (стр. 116).
3. Убедитесь, что файл создался в указанной папке.

Теперь откроем сохраненную пользовательскую палитру из файла.

1. Закройте окно проекта-примера, а затем вновь запустите в рантайм. Откройте окно выбора цвета: пользовательская палитра будет пуста.
2. Закройте окно выбора цвета и нажмите **Загрузить пользовательскую палитру**. Активируется событие ButtonPressed кнопки. Ознакомьтесь с обработчиком события. Экземпляр *Files* компонента **Окружение: файлы** извлечет содержимое файла `colors.json` и поместит его в компонент 2.2.6. Диалог выбора цвета (стр. 116) в качестве значения свойства CustomColors.
3. Откройте окно выбора цвета и убедитесь, что пользовательская палитра обновилась.

## 1.4.2. Подключение к серверу для просмотра дерева сигналов

ПРИМЕЧАНИЕ

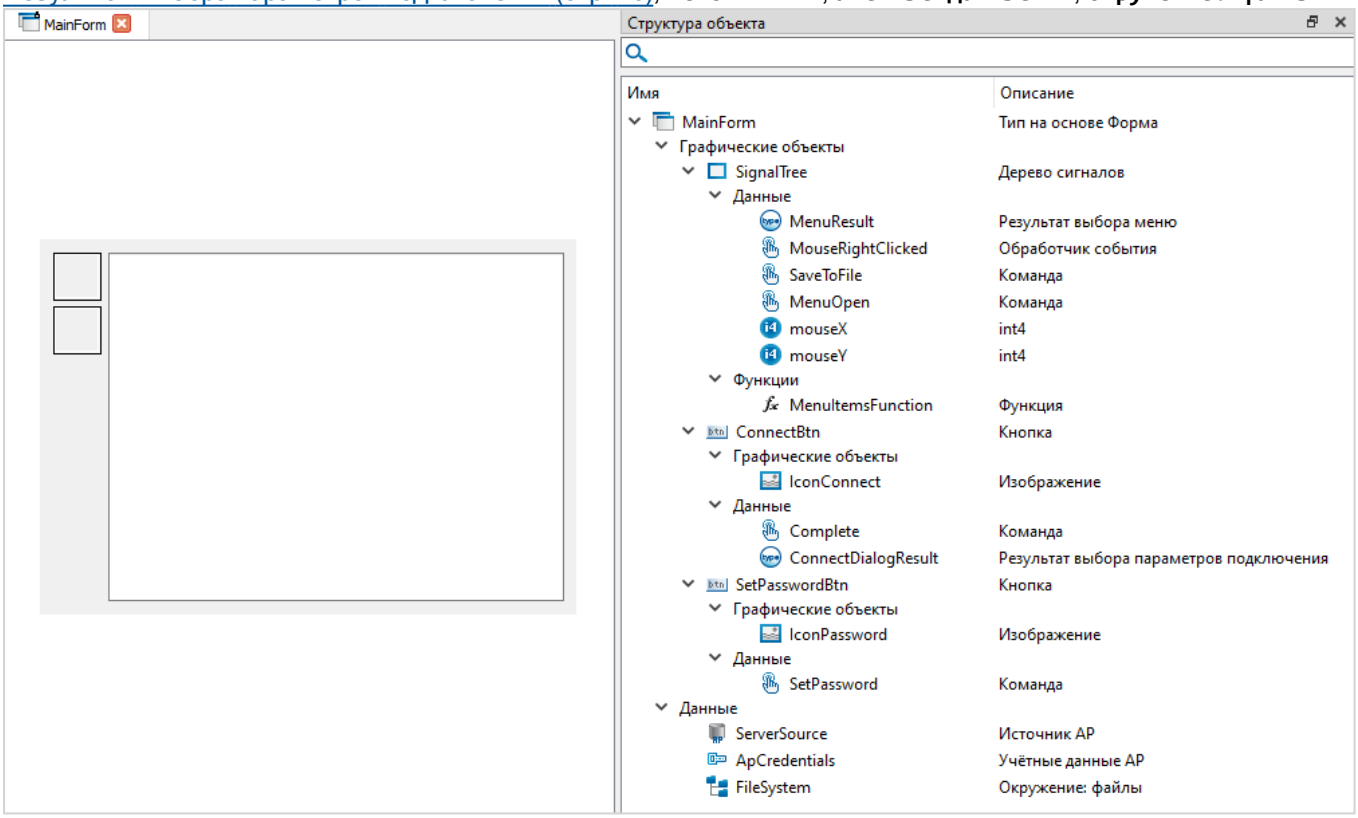
В разделе продемонстрировано применение компонентов 2.2.7. Диалог подключения к серверу (стр. 118), 2.2.10.3. Диалог ввода учетных данных (стр. 137), 2.1.16. Дерево сигналов (стр. 83).

Раздел демонстрирует работу приложения, которое позволяет подключиться к серверу для просмотра дерева сигналов.

Приложение реализовано в виде проекта-примера. Скачайте проект-пример и откройте в Дизайнер Alpha.HMI для наглядности.

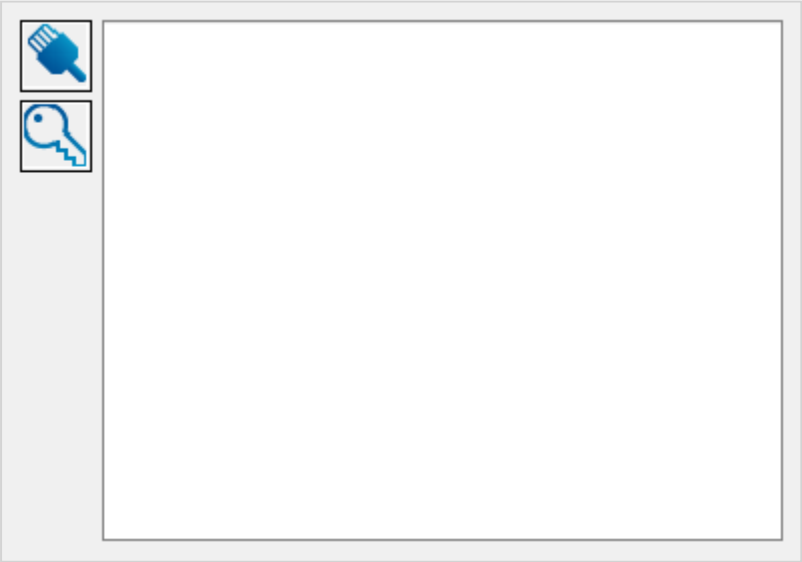
На главной форме проекта размещены кнопки подключения к серверу и ввода учетных данных сервера, поле вывода дерева сигналов. В качестве вспомогательных компонентов используются экземпляры компонентов

2.1.12.3. Результат выбора меню (стр. 62), 2.1.8. Обработчик события (стр. 36), **Команда**,**int4**, 2.1.15.4. Результат выбора параметров подключения (стр. 76), **Источник АР**, **Учетные данные АР**, **Окружение: файлы**.



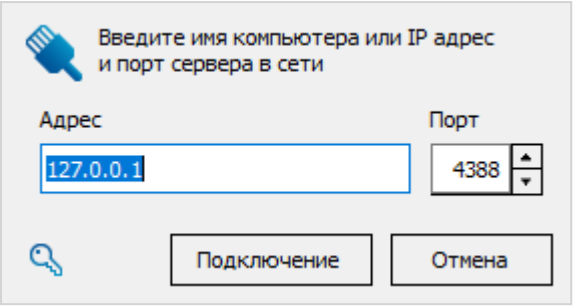

Запустите проект-пример в рантайм. Откроется окно приложения, в котором доступны:

- кнопка подключения к серверу;
- кнопка ввода учетных данных сервера;
- поле вывода дерева сигналов.



## Подключение к серверу

Для подключения к серверу нажмите . Активируется событие ButtonPressed объекта *ConnectBtn*, открывающее 2.2.7. Диалог подключения к серверу (стр. 118) в новом окне.




В диалоговом окне укажите адрес, порт сервера, учетные данные сервера и нажмите **Подключение**.

- Адрес, порт сервера и учетные данные сохраняются в экземпляр 2.1.15.4. Результат выбора параметров подключения (стр. 76) *ConnectDialogResult*, указанный в свойстве **pResult** компонента 2.2.7. Диалог подключения к серверу (стр. 118).
- Адрес и порт сервера из экземпляра 2.1.15.4. Результат выбора параметров подключения (стр. 76) передаются в **Источник АР** *ServerSource* по команде *Complete*. Ссылка на *Complete* указана в свойстве **OkAction** компонента 2.2.7. Диалог подключения к серверу (стр. 118).
- Учетные данные из экземпляра 2.1.15.4. Результат выбора параметров подключения (стр. 76) сохраняются в файл, откуда передаются в экземпляр *ApCredentials* типа **Учетные данные АР**. В свойстве **Источник учетных данных** экземпляра **Источник АР** указана ссылка на *ApCredentials*. После этого происходит попытка подключения к серверу.

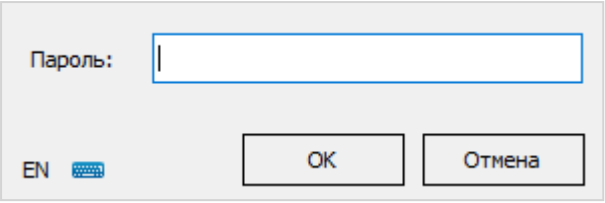
Ознакомьтесь с кодом обработчика события **Invoked** команды *Complete*, написанным на языке Alpha.Om.

```
// сохранение адреса и порта сервера
ServerSource.Location=ConnectDialogResult.Location;
ServerSource.Port=ConnectDialogResult.Port;
// сохранение в файл учетных данных
passwordFile: string = FileSystem.CombinePath(FileSystem.ProjectFolder,
"password.txt");
FileSystem.WriteTextFile(passwordFile, ConnectDialogResult.Credentials);
// подгрузка сохраненных значений в ApCredentials
ApCredentials.FilePath = passwordFile;
ApCredentials.Load();
// запуск подключения или сервера (если до этого он был выключен)
ServerSource.Active = true;
```

Учетные данные сервера можно вводить в отдельном диалоговом окне. Диалогом удобно пользоваться в следующих случаях:

- адрес и порт сервера уже указаны в свойствах экземпляра компонента **Источник АР**, и для подключения к серверу требуется ввести пароль;
- требуется изменить пароль, введенный в Диалоге подключения к серверу.

Для того чтобы ввести только учетные данные для подключения к серверу, нажмите . Активируется событие **ButtonPressed** объекта *SetPasswordBtn*, открывающее 2.2.10.3. Диалог ввода учетных данных (стр. 137) в новом окне.



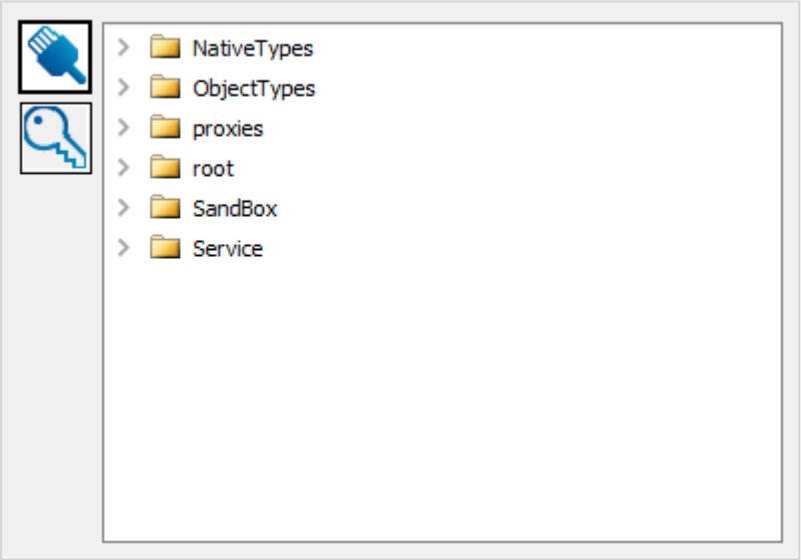

В диалоговом окне укажите пароль и нажмите **OK**.

- Введенный пароль передается в экземпляр *ApCredentials* типа **Учетные данные АР**.
- Активируется команда *SetPassword*, ссылка на которую указана в свойстве OkAction компонента 2.2.10.3. Диалог ввода учетных данных (стр. 137). Происходит попытка подключения к серверу.

  Ознакомьтесь с кодом обработчика события Invoked команды *SetPassword*, написанным на языке Alpha.Om.

```
ServerSource.Active = true;
```

В случае успешного подключения к серверу выводится дерево сигналов.




Рассмотрим использование свойств дерева сигналов на примере свойства pMouseRightClicked, активируемого при нажатии правой кнопкой мыши на узле дерева сигналов. В данном примере по щелчку правой кнопкой мыши на узле дерева сигналов открывается контекстное меню, которое позволяет сохранять в файл описание выбранного узла.

В свойстве pMouseRightClicked указывается ссылка на обработчик события *MouseRightClicked*. Для того чтобы контекстное меню открывалось рядом с выбранным узлом, координаты курсора передаются в экземпляры *mouseX* и *mouseY* типа **int4**. Выполняется команда *MenuOpen*, открывающая контекстное меню. Ознакомьтесь с кодом обработчика события *MouseRightClicked*, написанным на языке JavaScript.

```
var arg = JSON.parse(Args.Get());
mouseX = arg.ScreenX;
mouseY = arg.ScreenY;
MenuOpen.Invoke();
```

Для задания пунктов контекстного меню необходимо создать функцию и сослаться на нее в свойстве **Элементы меню** обработчика события **Invoked** команды *MenuOpen*. В данном примере создана функция **MenuItemsFunction**. Ознакомьтесь с кодом этой функции, написанным на языке JavaScript.

```
var items = [
{
'id': "txt",
'title': "Сохранить в файл"
}
];
return JSON.stringify(items);
```

При выборе пункта из списка контекстного меню:

- Результат выбора меню сохраняется в *MenuResult*.
- Активируется команда *SaveToFile*, сохраняющая описание выбранного узла в файл. Ознакомьтесь с кодом обработчика события **Invoked** этой команды, написанным на языке Alpha.Om.

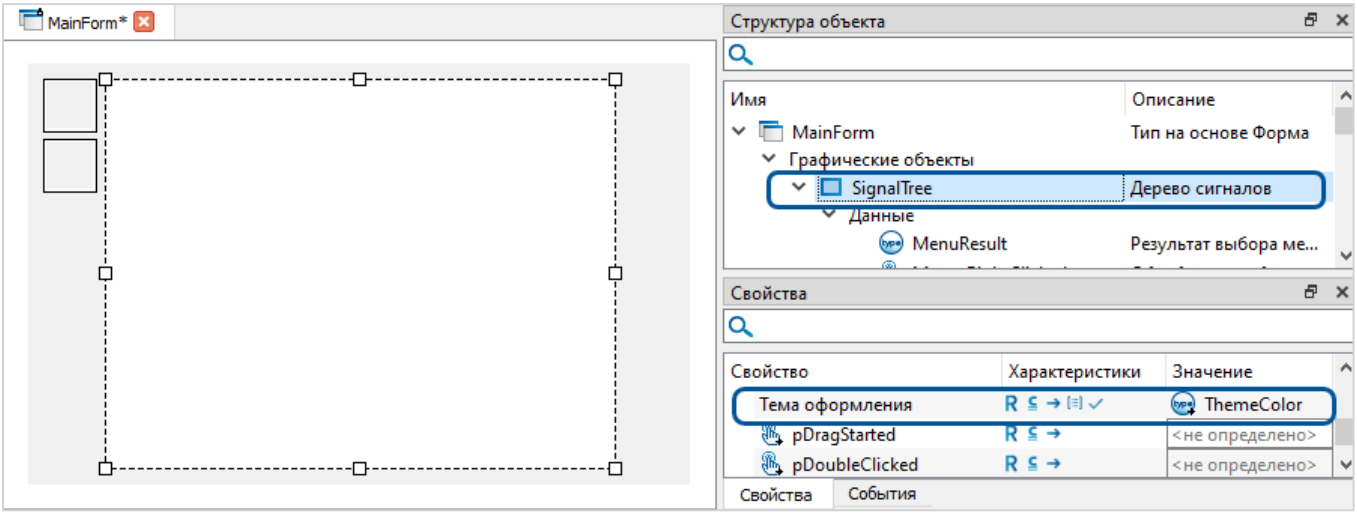
```
// сохранение в файл описания выбранного узла
itemFile: string = FileSystem.CombinePath(FileSystem.ProjectFolder,
"item.txt");
FileSystem.WriteTextFile(itemFile, SignalTree.GetCurrentItemInfo());
```

В результате происходит сохранение описания выбранного узла в файл. Файл сохраняется в папке `output`, расположенной в папке проекта.

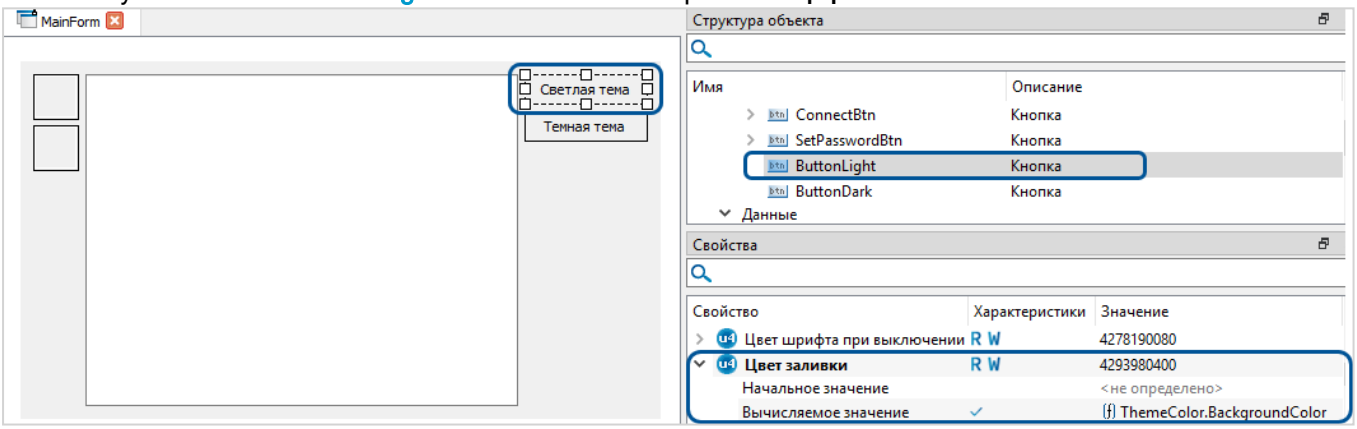
## 1.4.3. Изменение темы оформления

Рассмотрим изменение темы оформления на примере проекта приложения, описанного в разделе 1.4.2. Подключение к серверу для просмотра дерева сигналов (стр. 12).

Добавьте на форму экземпляр компонента 2.1.1. Тема оформления (стр. 18). Экземпляр не отображается на форме, а виден только в **Структуре объекта**. Экземпляр компонента **Тема оформления** хранит шрифты и цвета в виде значений свойств. Более подробно о свойствах можно прочитать в Справочном руководстве. Для всех экземпляров, цветовую схему которых требуется изменить, укажите в свойстве **Тема оформления** ссылку на экземпляр компонента **Тема оформления**. Эти экземпляры будут наследовать значения свойств экземпляра компонента **Тема оформления**.

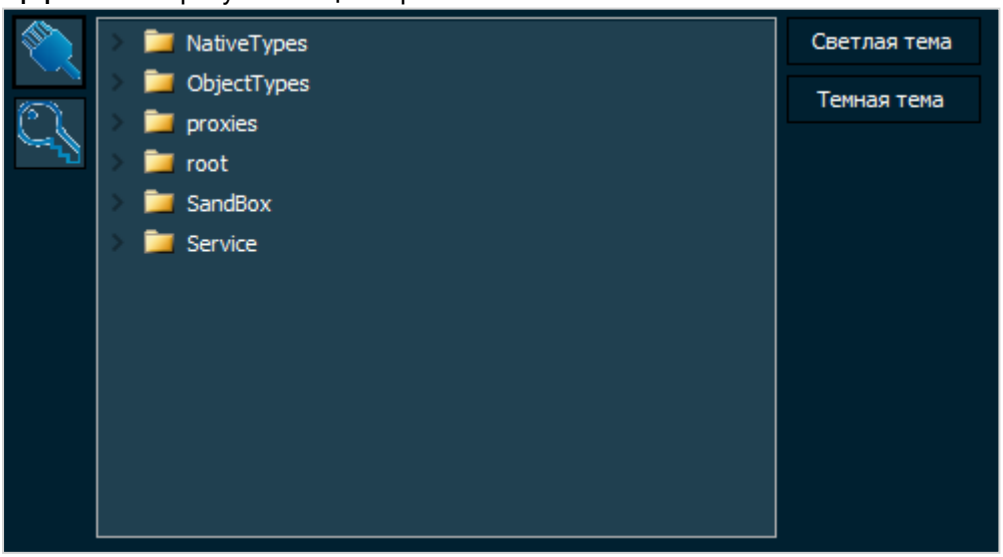



Если в экземпляре компонента не предусмотрено свойство Тема оформления, то для изменения значения цвета или шрифта выберите другое свойство экземпляра. Например, если нужно изменить цвет экземпляра компонента **Кнопка**, то для этого можно выбрать свойство Цвет заливки, где в качестве вычисляемого значения указать свойство BackgroundColor экземпляра **Тема оформления**.




В результате цвет экземпляра компонента **Кнопка** будет принимать значение, указанное в свойстве BackgroundColor экземпляра компонента **Тема оформления**.

Для изменения цветовой схемы приложения можно использовать готовые темы оформления. Для этого используйте функции, описанные в Справочном руководстве. Например, функция Dark() применит к приложению темную тему. Данная функция переопределит значения свойств экземпляра компонента **Тема оформления**. В результате цвет приложения изменится на темный.



# 2. Справочное руководство

## 2.1. Типы

### 2.1.1. Тема оформления

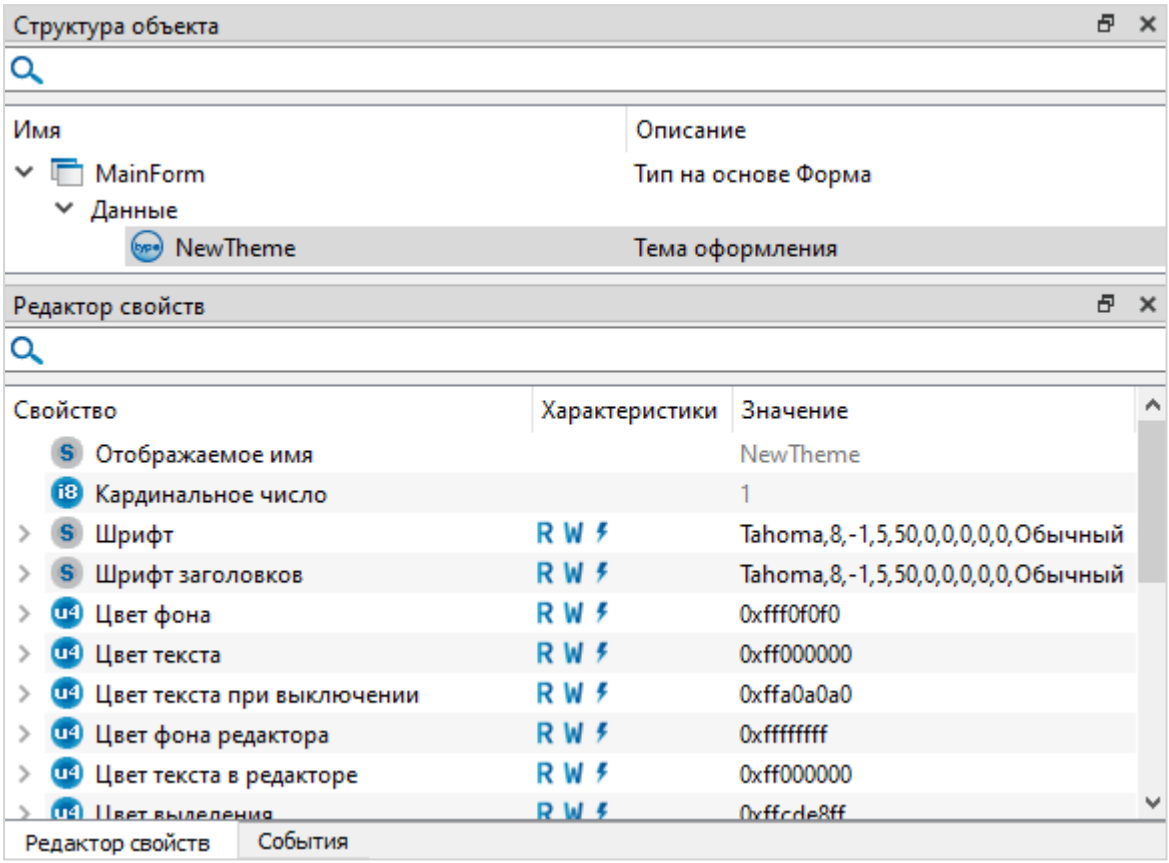
Компонент, позволяющий менять тему оформления визуальных компонентов Alpha.HMI.CommonLib: шрифты, цвет текста, фона и т.п.. Подробнее - в свойствах компонента.

Для оформления визуальных компонентов, добавленных на панель инструментов, используйте компонент 2.1.13.5. Тема оформления панели инструментов (стр. 68).

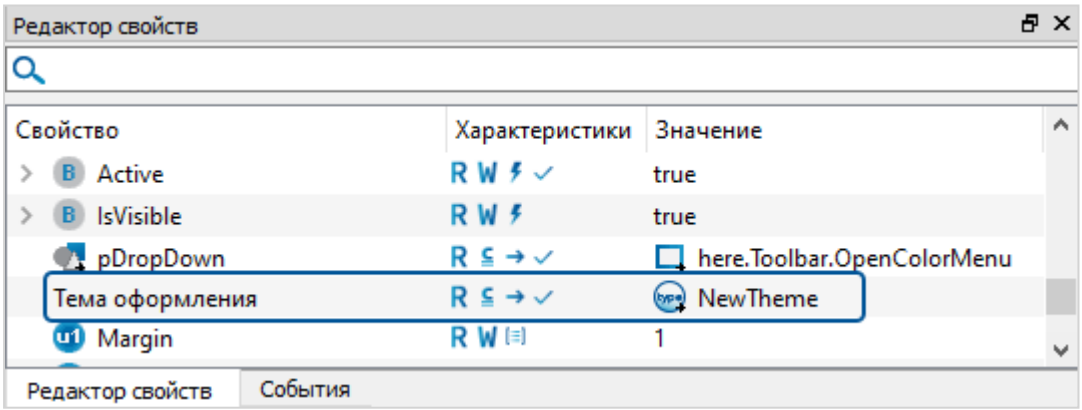
Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

ПРИМЕР

*NewTheme* - экземпляр типа **Theme**.




Чтобы применить тему *NewTheme* к объекту, укажите в свойстве 2.3.1. Тема оформления (стр. 137) этого объекта ссылку на экземпляр *NewTheme*.




ПРИМЕЧАНИЕ

На экземпляр может быть сделано сколько угодно ссылок.

## 2.1.1.1. Свойства

### Шрифт (Font)

Шрифт основного текста. Тип значения - string.

## Шрифт заголовков (TitleFont)

Шрифт заголовков. Тип значения - string.

## Цвет фона (BackgroundColor)

Цвет основной области компонента. Тип значения - уведомляющий uint4.

## Цвет текста (ForegroundColor)

Цвет текста в основной области компонента. Тип значения - уведомляющий uint4.

## Цвет текста при выключении (DisabledFontColor)

Цвет текста неактивных элементов. Тип значения - уведомляющий uint4.

## Цвет фона редактора (EditorBrushColor)

Цвет фона области компонента, в которой можно изменить значение. Тип значения - уведомляющий uint4.

## Цвет текста в редакторе (EditorFontColor)

Цвет текста в области компонента, в которой можно изменить значение. Тип значения - уведомляющий uint4.

## Цвет выделения (SelectionColor)

Цвет активного (выбранного) элемента в области компонента, в которой можно изменить значение. Тип значения - уведомляющий uint4.

## Цвет рамки (BorderColor)

Цвет рамки, ограничивающей область компонента, в которой можно изменить значение. Тип значения - уведомляющий uint4.

## Цвет рамки при выключении (DisabledBorderColor)

Цвет рамки, ограничивающей область компонента, в которой можно изменить значение, если эта область неактивна. Тип значения - уведомляющий uint4.

## Цвет фона панели инструментов (ToolbarBrushColor)

Цвет фона панели инструментов. Тип значения - уведомляющий uint4.

## Цвет текста на панели инструментов (ToolbarFontColor)

Цвет текста на панели инструментов. Тип значения - уведомляющий uint4.

### Цвет редактора на панели инструментов (ToolbarEditorColor)

Цвет фона расположенного на панели инструментов компонента, в котором можно изменить значение. Тип значения - уведомляющий uint4.

### Цвет выделения на панели инструментов (ToolbarSelectionColor)

Цвет выделения элемента на панели инструментов. Тип значения - уведомляющий uint4.

### Цвет фона меню (MenuBrushColor)

Цвет фона меню. Тип значения - уведомляющий uint4.

### Цвет текста в меню (MenuFontColor)

Цвет текста в меню. Тип значения - уведомляющий uint4.

### Цвет текста недоступного элемента меню (MenuDisabledFontColor)

Цвет текста недоступного элемента меню. Тип значения - уведомляющий uint4.

### Цвет выделенного элемента меню (MenuSelectionColor)

Цвет выделенного элемента меню. Тип значения - уведомляющий uint4.

## 2.1.1.2. Функции

### Light

```
void Presets.Light()
```

Изменяет значения свойств экземпляра **Тема оформления** (цвет фона, цвет текста, цвет элемента, цвет выделения, цвет рамок) в диапазоне от белого до светло-серого.

### Gray

```
void Presets.Gray()
```

Изменяет значения свойств экземпляра **Тема оформления** (цвет фона, цвет текста, цвет элемента, цвет выделения, цвет рамок) в диапазоне от белого до серого.

### Dark

```
void Presets.Dark()
```

Изменяет значения свойств экземпляра **Тема оформления** (цвет фона, цвет текста, цвет элемента, цвет выделения, цвет рамок) в диапазоне от белого до темно-синего.

## GetViewSettings

```
string GetViewSettings()
```

Возвращает настройки текущей темы в виде json-строки. Эти настройки могут использоваться для восстановления внешнего вида компонентов с помощью функции **UpdateView()** при перезапуске проекта.

ПРИМЕР

Пример возвращаемого значения:

```
{
 "Font":"Tahoma,8,-1,5,50,0,0,0,0,0,Обычный",
 "TitleFont":"Tahoma,8,-1,5,50,0,0,0,0,0,Обычный",
 "BackgroundColor":"0xfff0f0f0",
 "ForegroundColor":"0xff000000",
 "DisabledFontColor":"0xffa0a0a0",
 "EditorBrushColor":"0xffffffff",
 "EditorFontColor":"0xff000000",
 "SelectionColor":"0xffcde8ff",
 "BorderColor":"0xff808080",
 "DisabledBorderColor":"0xffa0a0a0",
 "ToolbarBrushColor":"0xfff0f0f0",
 "ToolbarFontColor":"0xff000000",
 "ToolbarEditorColor":"0xffffffff",
 "ToolbarSelectionColor":"0xffcde8ff",
 "MenuBrushColor":"0xfff0f0f0",
 "MenuFontColor":"0xff000000",
 "MenuDisabledFontColor":"0xffa0a0a0",
 "MenuSelectionColor":"0xffcde8ff"
}
```

## UpdateView

```
void UpdateView(string json)
```

Меняет внешний вид компонентов, у которых в свойстве 2.3.1. Тема оформления (стр. 137) указана текущая тема, в соответствии с настройками этой темы. Настройки могут быть получены в результате вызова функции **GetViewSettings()**.

Входной параметр - настройки текущей темы в виде json-строки.

# 2.1.2. Конвертер значений

Компонент, использующийся для различных преобразований значений.

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.
Чтобы выполнить преобразование, вызовите функцию компонента: Converter.Function().

## 2.1.2.1. Функции

### Convert

```
variant Convert(variant value, string dataType)
```

Конвертирует значение value типа variant в значение типа dataType. Результат функции хранится в переменной типа variant.

В качестве значения dataType можно указывать только названия типов данных языка Alpha.Om. Типы можно найти в документе на соответствующий продукт, либо в таблице, приведенной в описании функции CanonicalTypeToDataType (стр. 26) (см. столбец **Название типа**). Также можно использовать значение, возвращаемое этой функцией.

Входные параметры:

- конвертируемое значение;
- тип данных, в который конвертируется значение.

Преобразование выполняется с учетом ограничений пространства имен TypeConvert. Подробнее об ограничениях - в документе на Alpha.Om.

ПРИМЕР

```
number: variant = Converter.Convert(52, "uint4")
```

### StringToVariant

```
variant StringToVariant(string value, string dataType)
```

Конвертирует строковое значение value в значение типа dataType. Результат функции хранится в переменной типа variant.

В качестве значения dataType можно указывать только названия типов данных языка Alpha.Om. Типы можно найти в документе на соответствующий продукт, либо в таблице, приведенной в описании функции CanonicalTypeToDataType (стр. 26) (см. столбец **Название типа**). Также можно использовать значение, возвращаемое этой функцией.

Входные параметры:

- конвертируемое значение;
- тип данных, в который конвертируется значение.

ПРИМЕР

```
number: variant = Converter.StringToVariant("52", "uint4")
```

## VariantToString

```
string VariantToString(variant value, string dataType)
```

Конвертирует переменную типа variant, хранящую значение value типа dataType, в значение типа string.

В качестве значения dataType можно указывать только названия типов данных языка Alpha.Om. Типы можно найти в документе на соответствующий продукт, либо в таблице, приведенной в описании функции CanonicalTypeToDataType (стр. 26) (см. столбец **Название типа**). Также можно использовать значение, возвращаемое этой функцией.

Входные параметры:

- конвертируемое значение;
- тип данных конвертируемого значения.

ПРИМЕР

```
Number.Text = Converter.VariantToString(52, "uint4")
```

## VariantToLocalizedString

```
string VariantToLocalizedString(variant value, string dataType)
```

Конвертирует переменную типа variant, хранящую значение value типа dataType, в значение типа string с учетом региональных настроек. В региональных настройках могут быть заданы точность (количество знаков после разделителя) и знак разделителя.

В качестве значения dataType можно указывать только названия типов данных языка Alpha.Om. Типы можно найти в документе на соответствующий продукт, либо в таблице, приведенной в описании функции CanonicalTypeToDataType (стр. 26) (см. столбец **Название типа**). Также можно использовать значение, возвращаемое этой функцией.

Входные параметры:

- конвертируемое значение;
- тип данных конвертируемого значения.

ПРИМЕР

```
Text_1 = String.VariantToLocalizedString(a, "double");
```

## ValidateValue

```
string ValidateValue(string value, string dataType)
```

Проверяет значение value, записанное в виде строки, на соответствие указанному типу данных dataType.

Входные параметры:

- проверяемое значение;
- проверяемый тип данных.

В качестве значения dataType можно указывать только названия типов данных языка Alpha.Om. Типы можно найти в документе на соответствующий продукт, либо в таблице, приведенной в описании функции CanonicalTypeToDataType (стр. 26) (см. столбец **Название типа**). Также можно использовать значение, возвращаемое этой функцией.

Возвращает:

- текст ошибки в случае ее возникновения;
- пустую строку, если ошибки не возникает.

ПРИМЕР

Вызов: `Converter.ValidateValue("1234", "uint4")`

## ValidateRange

```
string ValidateRange(variant value, string dataType, variant min, variant max)
```

Проверяет, входит ли указанное значение value типа variant в допустимый диапазон от min до max.

Входные параметры:

- проверяемое значение;
- тип данных проверяемого значения;
- нижняя граница диапазона min;
- верхняя граница диапазона max.

Если не указать значения min и/или max, будут использоваться границы диапазона указанного типа данных.

В качестве значения dataType можно указывать только названия типов данных языка Alpha.Om. Типы можно найти в документе на соответствующий продукт, либо в таблице, приведенной в описании функции CanonicalTypeToDataType (стр. 26) (см. столбец **Название типа**). Также можно использовать значение, возвращаемое этой функцией.

Возвращает:

- текст ошибки в случае ее возникновения;
- пустую строку, если ошибки не возникает.

ПРИМЕР

Вызов: `Converter.ValidateRange(123, "uint8", 1, 500)`

## ValidateValueRange

```
string ValidateValueRange(string value, string dataType, variant min, variant max)
```

Проверяет значение value, указанное в виде строки, на соответствие указанному типу данных dataType и на принадлежность допустимому диапазону от min до max.

Входные параметры:

- проверяемое значение;
- проверяемый тип данных;
- нижняя граница диапазона min;
- верхняя граница диапазона max.

В качестве значения dataType можно указывать только названия типов данных языка Alpha.Om. Типы можно найти в документе на соответствующий продукт, либо в таблице, приведенной в описании функции CanonicalTypeToDataType (стр. 26) (см. столбец **Название типа**). Также можно использовать значение, возвращаемое этой функцией.

Возвращает:

- текст ошибки в случае ее возникновения;
- пустую строку, если ошибки не возникает.

ПРИМЕР

Вызов: `Converter.ValidateValueRange(123, "uint8", 1, 500)`

## CanonicalTypeToDataType

```
string CanonicalTypeToDataType(uint1 canonicalType)
```

Вспомогательная функция. Возвращает название канонического типа данных, указанного в числовом виде. Возвращаемое значение можно использовать в качестве входного параметра dataType остальных функции **Convert**.

Входной параметр - число, соответствующее каноническому типу данных.

**Таблица соответствия численного и текстового названий канонического типа данных**

| Число типа | Название типа |
|---|---|
| 0 | "void" |
| 1 | "bool" |
| 2 | "int1" |
| 3 | "uint1" |
| 4 | "int2" |
| 5 | "uint2" |
| 6 | "int4" |
| 7 | "uint4" |

| Число типа | Название типа |
|---|---|
| 8 | "int8" |
| 9 | "uint8" |
| 10 | "float" |
| 11 | "double" |
| 12 | "timestamp" |
| 14 | "string" |

ПРИМЕР

Вызов: `a = Converter.CanonicalTypeToDataType(3)`

Результат: переменная a типа string, значение - **«uint1»**.

## QualityCodeToString

```
string QualityCodeToString(uint4 quality)
```

Возвращает текстовое описание качества сигнала на основе числового кода качества сигнала.

Входной параметр - числовой код качества сигнала.

Описание состоит из трех значений:

- качество сигнала;
- числовой код;
- дополнительное описание.

ПРИМЕР

Код качества сигнала можно получить с помощью внутреннего аргумента или свойства экземпляра компонента **Элемент AP <type>**. Полученное значение можно использовать в качестве входного параметра функции QualityCodeToString().

Пример возвращаемого описания качества сигнала:

**«хорошее: 193 - Good (Low)»**

На основании возвращаемого значения, о сигнале можно сделать выводы:

- сигнал имеет хорошее качество;
- значение получено от источника с хорошим качеством;
- качество сигнала усечено по нижней границе пересчета.

Описания числовых кодов качества сигнала можно найти в разделе "Качество сигналов" OPC DA Client модуля OPC DA документации на Alpha.Server. Более подробно о значениях дополнительного описания можно почитать в разделах "Сигналы Alpha.Server" и "Логическая обработка данных" документации на Alpha.Server.

### GetVariantDataType

```
string GetVariantDataType(variant value)
```

Возвращает тип данных переменной value в виде строки. Если тип данных определить не удалось, то возвращается пустая строка.

Входной параметр - переменная типа variant.

## 2.1.3. Утилиты для работы со строками

Компонент, использующийся для различных преобразований строковых значений.

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

Чтобы выполнить преобразование, вызовите функцию компонента: StringUtils.Function().

## 2.1.3.1. Функции

### GetDeclension

```
string GetDeclension(int8 number, string nominative, string genitive, string plural)
```

Склоняет указанное во входных параметрах слово в зависимости от указанного числа (количества).

Входные параметры:

- число (количество), в зависимости от которого склоняется слово;
- склоняемое слово в именительном падеже;
- склоняемое слово в родительном падеже;
- склоняемое слово во множественное числе.

Возвращает слово в склонении, подходящем указанному числу.

ПРИМЕР

Предположим, некая переменная hour хранит количество часов, оставшихся до конца сессии пользователя, а в переменную word нужно записывать склоняемое слово "час". Тогда:

```
word: string = StringUtils.GetDeclension(hour, "час", "часа", "часов");
Message.Text = "Осталось " + hour + word;
```

### CalcRowHeight

```
int4 CalcRowHeight(string font)
```

Вычисляет примерную высоту строки текста для указанного шрифта font. Применяется для определения высоты заголовка в таблице при изменении шрифта темы 2.1.1. Тема оформления (стр. 18).

Входным значением является строка, в которой перечислены параметры шрифта.

ПРИМЕР

Пример строки входного значения:

```
Tahoma,8,-1,5,50,0,0,0,0,0,Обычный
```

## 2.1.4. Утилиты для работы с окном

Вспомогательный тип для работы с окнами (формами в режиме исполнения).

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

### 2.1.4.1. Свойства

#### Ссылка на форму (pForm)

Ссылка на форму, поведение окна которой настраивается с помощью функций этого компонента.

### 2.1.4.2. Функции

#### FitFormSize

```
void FitFormSize(double windowWidth, double windowHeight)
```

Подстраивает размер формы под размер окна, в котором она открывается, с учетом масштаба формы.

Входные параметры - ширина и высота окна.

#### CorrectPosition

```
void CorrectPosition()
```

Корректирует положение окна так, чтобы оно отобразилось на первом мониторе, если:

- окно не отображается ни на одном из имеющихся мониторов;
- окно невозможно переместить из-за выхода заголовка окна за пределы монитора.

Применяется при открытии окна с определенными координатами, которые могут выходить за область текущего монитора.

## FitToMonitor

```
void FitToMonitor(int4 monitor)
```

Размещает окно на указанном мониторе. Входной параметр - номер монитора, который можно получить с помощью функции GetMonitor().

## CorrectPositionAboutPoint

```
void CorrectPositionAboutPoint(int4 x, int4 y, int4 pointHeight)
```

Корректирует положение окна относительно указанной точки так, чтобы окно располагалось в пределах монитора. Применяется при открытии контекстного меню и других всплывающих окон.

Входные параметры:

- X-координата точки;
- Y-координата точки;
- смещение вверх относительно низа монитора.

## GetMonitor

```
int4 GetMonitor()
```

Возвращает номер монитора, на котором отображается большая часть окна. Если окно не отображается ни на одном из имеющихся мониторов, то функция возвращает значение «**-1**».

Можно использовать совместно с функцией FitToMonitor(), чтобы расположить окно целиком на том мониторе, где открывается его большая часть.

## GetMonitorForRect

```
int4 GetMonitorForRect(double left, double top, double width, double height)
```

Возвращает номер монитора, на котором отображается большая часть родительского компонента. Может использоваться для открытия формы в центре определенного прямоугольника.

Входные параметры:

- X-координата левого верхнего угла родительского компонента;
- Y-координата левого верхнего угла родительского компонента;
- ширина родительского компонента;
- высота родительского компонента.

### FitToRect

```
void FitToRect(double left, double top, double width, double height)
```

Размещает форму в центре заданного прямоугольника.

Входные параметры:

- X-координата левого верхнего угла родительского компонента;
- Y-координата левого верхнего угла родительского компонента;
- ширина родительского компонента;
- высота родительского компонента.

Для того чтобы вызвать функцию в обработчике события AboutToOpen какой-либо формы, в свойстве формы Положение окна укажите значение **«Вручную»**.

## 2.1.5. Утилиты для работы с файлами

Компонент, использующийся для работы с файлами.

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

### 2.1.5.1. Свойства

#### Произошла ошибка (pErrorOccurred)

Ссылка на экземпляр типа 2.1.8. Обработчик события (стр. 36), вызываемый при возникновении ошибки.

#### Описание последней ошибки (LastError)

Текст ошибки последней операции.

ПРИМЕР

Текст ошибки при открытии несуществующего файла:

```
Не получается открыть для чтения файл 'C:\Users\test.txt'
Не удается найти указанный файл.
```

ПРИМЕЧАНИЕ

Для получения описания последней ошибки можно обращаться напрямую к объекту **LastError** или к его свойству Value.

### 2.1.5.2. Функции

#### ReadFromFile

```
string ReadFromFile(string filePath)
```

Предоставляет содержимое текстового файла в виде строкового значения.

Входной параметр - полный путь к файлу.

#### WriteToFile

```
void WriteToFile(string filePath, string text)
```

Выполняет запись текста в файл.

Входные параметры - полный путь к файлу, текст.

## 2.1.6. Информация о продукте

Вспомогательный тип. Содержит информацию о продукте, компании-разработчике и платформе. Позволяет получить эту информацию при подключении продукта к проекту Alpha.HMI в качестве внешнего модуля.

При разработке собственного продукта экземпляр типа следует помещать в глобальные объекты проекта.

### 2.1.6.1. Свойства

#### CompanyName

Название компании-разработчика продукта. Тип значения - string.

#### CompanyTag

Тег компании. Тип значения - string.

#### PlatformName

Название платформы. Тип значения - string.

#### PlatformTag

Тег платформы. Тип значения - string.

### FamilyName

Название семейства продуктов. Тип значения - string.

### FamilyTag

Тег семейства продуктов. Тип значения - string.

### ProductName

Название продукта. Тип значения - string.

### ProductTag

Тег продукта. Тип значения - string.

### FolderName

Имя папки для хранения данных продукта. Обычно совпадает с тегом компании. Тип значения - string.

### Version

Версия продукта. Тип значения - string.

## 2.1.6.2. Функции

### GetProgramDataFolder

```
string GetProgramDataFolder()
```

Предоставляет полный путь к папке, предназначенной для хранения данных программы - например, конфигурационных файлов, бэкапов и пр. Путь формируется из значений свойств:

```
Files.CombinePath(Files.AllUsersFolder, Files.CombinePath(FolderName,
ProductName))
```

где `FolderName` и `ProductName` - свойства типа.

### GetUserDataFolder

```
string GetUserDataFolder()
```

Предоставляет полный путь к папке, предназначенной для хранения пользовательских данных - например, настроек вида программы для конкретного пользователя. Путь формируется из значений свойств:

```
Files.CombinePath(Files.CurrentUserFolder, Files.CombinePath(FolderName,
ProductName))
```

где `FolderName` и `ProductName` - свойства типа.

## 2.1.7. Буфер аргументов события

Вспомогательный тип. Представляет собой буфер, предназначенный для передачи аргументов события, реализованного с помощью компонента **`Команда (Action)`**.

Запись и извлечение данных происходит по принципу FIFO[1] (очереди). При переполнении буфера новые аргументы вытесняют старые также по принципу очереди.

### Пример использования

Для демонстрации использования компонента разработан проект-пример. Проект-пример можно скачать и открыть в Дизайнер Alpha.HMI. Для работы с проектом-примером потребуются компоненты, указанные в 1.2. Подготовка к работе (стр. 7), и Alpha.Server с подключенным модулем логики.

Предположим, необходимо получить данные о сигналах из Alpha.Server в проекте Alpha.HMI посредством JSON-запросов к API сервера. В результате получения необходимых данных должна выполняться определенная команда. Запрос по каждому сигналу происходит отдельно. Таких запросов может быть достаточно много. Поскольку запросы происходят асинхронно, к началу обработки результатов может быть получено несколько ответов. Чтобы не потерять результаты запросов, каждый из них помещается в буфер типа **`EventArgs`**, который по очереди передает полученные данные в команду в качестве аргумента.

Откройте форму **`MainForm`** в проекте-примере. На форме размещена кнопка *`Получить ID (ButtonGetIDs)`*. Ознакомьтесь с кодом обработчика события ButtonPressed кнопки.

```
//по нажатию кнопки вызывается функция GetID типа ServerAPI
ServerAPI.GetID("Service.State.Pair");
ServerAPI.GetID("Service.State.Server");
ServerAPI.GetID("Service.Config.Version");
```

По нажатию кнопки вызывается функция GetID() компонента *`ServerAPI`*, разработанного для данного проекта-примера. В качестве входных параметров указаны имена сигналов Alpha.Server. Поскольку такие - сервисные - сигналы присутствуют в любой конфигурации, они выбраны для этого примера. Здесь можно указать любые имена сигналов, присутствующих в вашем сервере.

Обратите внимание на команду *`OnGetID`* здесь же, в компоненте *`ServerAPI`*. Это команда, выполняемая при получении ID сигнала из сервера. Она записывает полученное значение в текстовое поле *`TagsIDs`*. Именно в эту команду необходимо передавать аргумент - полученный ID сигнала.

Откройте тип **`ServerAPI`**. В типе созданы:

- функция GetID(), отправляющая JSON-запрос на получение ID по имени сигнала на сервер;
- элемент *`InvokeFromJSON`*, связанный с динамическим сигналом сервера **`InvokeFromJSON`**, куда функция GetID() отправляет JSON-запрос, а сервер помещает JSON-ответ;

[1] first-in-first-out - "первым пришел, первым ушел"

- *pOnGetID* - ссылка на команду *OnGetID*, реализованную в **MainForm**, выполняемую в результате получения ID сигнала;
- буфер *OnGetIDArgs*, предназначенный для передачи полученного ID из ссылки на команду *pOnGetID* в команду *OnGetID*. Размер буфера ограничен десятью записями. Размер буфера можно изменить в свойстве BufferSize.

Ознакомьтесь с функцией GetID(). Функция использует API предоставления данных сервера. Подробнее API и работа с ним описаны в документе на модуль логики Alpha.Server.

```
transac += 1;
//наращиваем номер транзации (обращения к серверу)
request: string = "{\"transaction\":\""+String.ToString
(transac)+"\",\"request\":
{\"target\":\"Service.ServerApi\",\"method\":\"GetIdByTagName\",\"input\":
{\"tagname\":\""+tagname+"\"}}}";
//в переменной request формируется запрос в формате JSON
InvokeFromJSON.TrySetValue(request);
//запрос помещается в компонент, связанный с динамическим сигналом сервера
InvokeFromJSON
//сервер помещает ответ в формате JSON в этот же сигнал
```

В случае успешной записи ответа сервера в компонент *InvokeFromJSON* активируется событие OnValueChanged этого компонента. Ознакомьтесь с обработчиком этого события.

```
var ask = JSON.parse(value);
//при изменении значения сигнала - то есть, при получении ответа от сервера -
//полученное значение записывается в переменную ask, где его необходимо
разобрать
var res = ask.result.return;
//в переменную res записывается значение ключа "return" - это и есть ID сигнала
OnGetIDArgs.Push(res);
//в буфер типа EventArgs записывается полученное значение - ID (иначе -
аргумент)
pOnGetID.Invoke();
//вызывается команда, в которую передается полученный аргумент
```

Полученный от сервера JSON-ответ необходимо разобрать с помощью функции JSON.parse(), и затем выделенный из ответа ID поместить в буфер *OnGetIDArgs* с помощью функции Push(). После этого с помощью функции Invoke() по ссылке *pOnGetID* вызывается команда *OnGetID*, куда полученный ID передается через буфер.

ОБРАТИТЕ ВНИМАНИЕ

Здесь разбирается внутренний аргумент события OnValueChanged компонента *InvokeFromJSON* - value, а не значение свойства Value этого компонента. Это тоже связано с асинхронностью процессов получения ответов от сервера.

Значение свойства Value перезаписывается при получении каждого нового значения, а в аргументе value полученное значение сохраняется до передачи в обработчик события. Так как процесс обработки асинхронный, то к моменту выполнения обработчика значение свойства Value может несколько раз измениться. А значение аргумента value будет тем же, что и в момент наступления события.

Вернитесь на форму **MainForm** и ознакомьтесь с обработчиком события **Invoked** команды *OnGetID*.

```
var getresult = ServerAPI.OnGetIDArgs.Get();
//переменная getresult создается для извлечения аргумента события из буфера
TagsIDs.Text = TagsIDs.Text + " " + getresult;
//значение переменной записывается в текстовое поле
```

Здесь с помощью функции **Get()** происходит извлечение аргумента события из буфера *OnGetIDArgs*. Затем это значение используется для записи в текстовое поле на форме. Таким образом из буфера извлекается каждый полученный ID в момент получения ответа от сервера.

## 2.1.7.1. Свойства

### Размер буфера (BufferSize)

Размер буфера аргументов, измеряемый количеством записей. Тип значения - int4.

## 2.1.7.2. Функции

### Push

```
void Push(string args)
```

Записывает аргумент в буфер. Запись помещается в конец очереди.

### Get

```
string Get()
```

Извлекает первый в очереди аргумент из буфера.

## 2.1.8. Обработчик события

Компонент позволяет создать пользовательское событие с возможностью передачи его аргументов. Реализован на базе компонента **Команда (Action)** и буфера аргументов **Args** типа 2.1.7. Буфер аргументов события (стр. 34).

Компонент можно использовать, например, когда необходимо, чтобы аргументы события, произошедшего на одной форме проекта, были переданы на другую форму в момент наступления события. Для этого:

1. На форме, ожидающей передачи аргументов события, создается экземпляр обработчика события *EventHandler*, а на форме, где ожидается наступление события и генерация его аргументов - ссылка на обработчик события *pEventHandler*.

2. Когда нужно уведомить *EventHandler* о наступлении события:
   2.1. В экземпляр *pEventHandler* помещается аргумент события. Для этого используется вложенный объект *pEventHandler.Args*. Аргумент помещается в буфер с помощью команды:

```
pEventHandler.Args.Push(string arg)
```

   2.2. Затем вызывается функция **Invoke()** экземпляра *pEventHandler*, активирующая событие *EventHandler*.
3. В обработчике события **Invoked** экземпляра *EventHandler*:
   3.1. Сначала извлекается переданный аргумент с помощью команды:

```
EventHandler.Args.Get()
```

   3.2. Затем выполняется обработка полученного аргумента.

Для корректной работы буфера с аргументами следует соблюдать следующие правила:

- один раз вызывать функцию **Push()** перед вызовом функции **Invoke()** экземпляра *pEventHandler* при формировании события;
- один раз вызывать функцию **Get()** при получении аргументов события в обработчике *EventHandler*.

## 2.1.8.1. Свойства

### Args

Объект, представляющий собой буфер аргументов события. Тип объекта - 2.1.7. Буфер аргументов события (стр. 34).

Запись и извлечение данных в буфер происходит по принципу FIFO[1] (очереди). При переполнении буфера новые аргументы вытесняют старые также по принципу очереди.

Имеет собственные свойства и функции, аналогичные свойствам и функциям компонента 2.1.7. Буфер аргументов события (стр. 34).

### Включено (Enabled)

Включает или отключает команду. Тип значения - bool.

Значения:

- **«true»** - команда включена и может выполняться;
- **«false»** - команда отключена и не может выполняться.

### Триггер (InvokeTrigger)

Свойство-триггер команды. Тип значения - bool. Значение по умолчанию - **«false»**, при записи значения **«true»** команда выполняется.

[1] first-in-first-out - "первым пришел, первым ушел"

### Размер буфера аргументов (ArgsBufferSize)

Размер буфера аргументов, измеряемый количеством записей. Тип значения - int4.

## 2.1.8.2. Функции

### Invoke

```
void Invoke()
```

Запускает пользовательскую команду, указанную в обработчике события Invoked.

## 2.1.8.3. События

### Invoked

Активируется в результате вызова функции Invoke() или записи значения «**true**» в свойство InvokeTrigger.

### EnabledChanged

Активируется в момент смены состояния команды, указанного в значении свойства Включено (Enabled).

## 2.1.9. Пользовательская команда

Компонент позволяет хранить описание команды и сочетание "горячих" клавиш в одном месте. Реализован на базе компонента **Команда**.

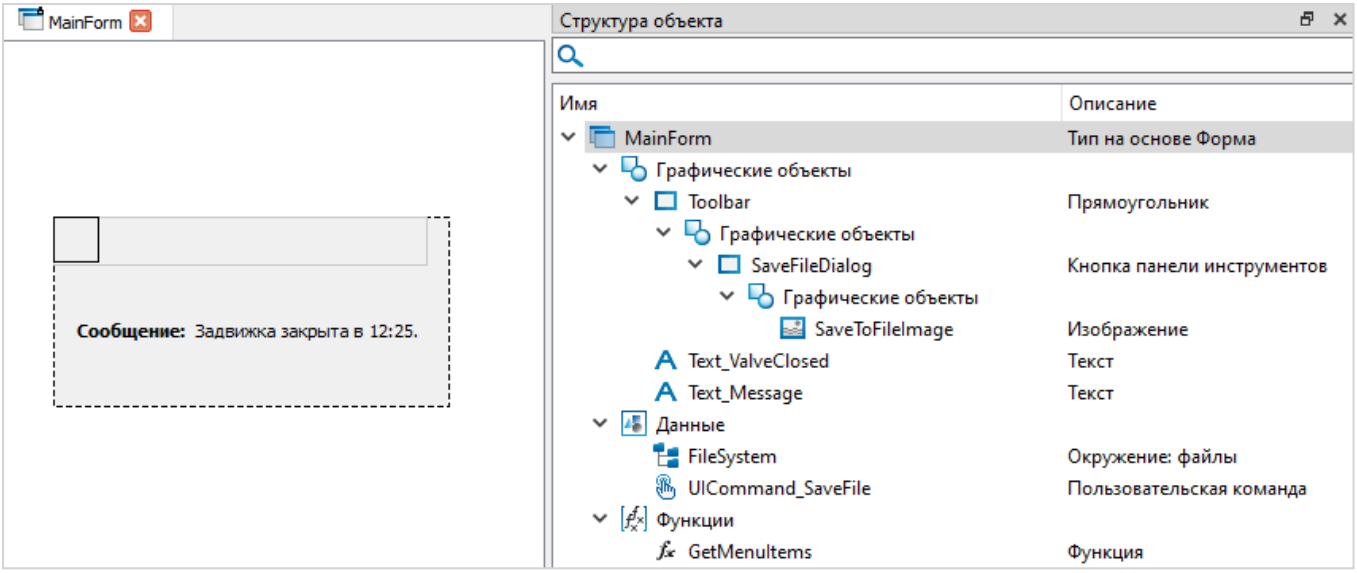
ПРИМЕЧАНИЕ

Обработка горячих клавиш настраивается в событии KeyPress формы. Подробнее об этом событии можно прочитать в Справочном руководстве на Alpha.HMI → Экранные формы → События → KeyPress.

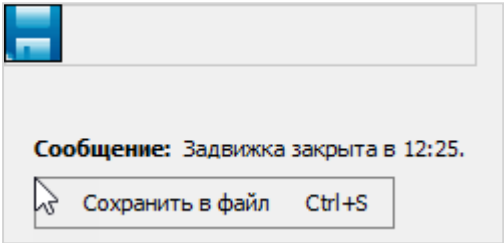
### Пример использования в проекте

Пусть необходимо разработать приложение, в котором можно сохранить текст в файл несколькими способами: нажав кнопку на панели инструментов, выбрав пункт контекстного меню формы или нажав сочетание "горячих" клавиш. Для этих целей можно использовать экземпляр типа **Пользовательская команда**. Описание команды и "горячих" клавиш будут храниться в его свойствах, сохранение содержимого экземпляра типа **Текст** будет выполняться по активации обработчика события экземпляра этого типа. Экземпляр типа **Пользовательская команда** можно вызвать в обработчиках разных событий.

На главной форме проекта разместите экземпляры типов 2.1.13.1. Кнопка панели инструментов (стр. 64) и **Текст**. В качестве вспомогательных компонентов используйте экземпляры типов **Прямоугольник**, **Изображение**, **Окружение: файлы** и 2.1.9. Пользовательская команда (стр. 38).




Тогда в рантайме приложение будет выглядеть так:




## Создание пользовательской команды

Перенесите экземпляр типа **Пользовательская команда** из **Библиотеки компонентов** на форму. Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**. Теперь можно перейти к настройкам этого экземпляра типа.

В экземпляре типа **Пользовательская команда**:

- в свойстве **Описание** укажите заголовок команды, который будет отображаться в контекстном меню;
- в свойстве **Сочетание клавиш** укажите сочетание "горячих" клавиш, по нажатию которых можно вызвать команду. Значение этого свойства будет отображаться в контекстном меню справа от описания команды;

› в обработчике события Invoked настройте сохранение содержимого экземпляра типа **Текст** в файл. Для этого можно использовать код, написанный на языке Alpha.Om:

```
//сохранение в файл
itemFile: string = FileSystem.CombinePath(FileSystem.ProjectFolder,
"text.txt");
FileSystem.WriteTextFile(itemFile, Text_ValveClosed.Text);
```

В результате вызова этого обработчика события в переменную **«itemFile»** типа string будет записан путь к файлу **«text.txt»**, хранящемуся в папке `output` проекта. В этом файле сохранится содержимое экземпляра типа **Текст**.

ПРИМЕЧАНИЕ

Если файла **«text.txt»** не существует в папке `output`, то он будет создан после первого исполнения кода.

## Вызов нажатием кнопки на панели инструментов

Команду можно вызвать по нажатию кнопки на панели инструментов. Для начала создайте панель инструментов:

1. Добавьте на форму экземпляр типа **Прямоугольник** - он будет выступать в качестве панели инструментов.
2. В экземпляре типа **Прямоугольник** разместите экземпляр типа **Кнопка панели инструментов**.

Чтобы по нажатию кнопки на панели инструментов вызвать команду, перейдите к обработчику события MouseClick экземпляра типа **Кнопка панели инструментов**. В этом обработчике используйте код, написанный на языке Alpha.Om:

```
UICommand_SaveFile.Invoke();
```

В результате будет активирована функция Invoke() экземпляра типа **Пользовательская команда**. Эта функция активирует обработчик события Invoked экземпляра типа **Пользовательская команда**, который сохранит содержимое экземпляра типа **Текст** в файл.

## Вызов в контекстном меню

Команду можно вызвать в контекстном меню формы. В этом меню в качестве заголовка пункта меню и сочетания "горячих" клавиш используйте значения свойств экземпляра типа **Пользовательская команда**, а в качестве ссылки на команду, выполняемую при выборе пункта меню - ссылку на сам экземпляр типа **Пользовательская команда**.

Элементы меню задаются в виде JSON-массива. О структуре этого массива можно почитать в свойстве Элементы меню (стр. 107). Массив можно описать с помощью функции. Чтобы создать функцию, в контекстном меню формы выберите **Создать функцию**.

Ознакомьтесь с кодом созданной функции GetMenuItems(), написанным на языке JavaScript:

```
var items = [
{
```

```
 'id': "save",
 'title': UICommand_SaveFile.Description.Value,
 'keyGesture': UICommand_SaveFile.KeyGesture.Value
}
];
return JSON.stringify(items);
```

Чтобы контекстное меню открывалось по нажатию правой кнопки мыши на форме, перейдите к событию MouseRightClick этой формы. В обработчике этого события откройте экземпляр типа **Меню** в новом окне. Настройте свойства **Меню**:

- в качестве начального значения свойства **X окна** используйте формулу `here.ScreenX(here.MouseX,0)`;
- в качестве начального значения свойства **Y окна** используйте формулу `here.ScreenY(0,here.MouseY)`. Использование функций ScreenX и ScreenY позволяет открывать контекстное меню в месте щелчка правой кнопкой мыши на форме. Подробнее о функциях ScreenX и ScreenY можно почитать в документации на Alpha.HMI → Справочное руководство → Экранные формы → Функции;
- в свойстве Элементы меню в виде формулы укажите ранее созданную функцию GetMenuItems();
- в свойстве pAction укажите ссылку на экземпляр типа **Пользовательская команда**. Тогда по нажатию пункта меню будет активирован обработчик события Invoked этого экземпляра.

## Вызов "горячими" клавишами

Команду можно вызвать по нажатию "горячих" клавиш. Пусть для сохранения содержимого экземпляра типа **Текст** в файл используется сочетание клавиш **«Ctrl»+«S»**.

Чтобы вызвать команду по нажатию этих "горячих" клавиш, перейдите к событию KeyPress формы. В обработчике этого события используйте код, написанный на языке Alpha.Om:

```
if (event.Modifiers == 0x4000000) // Ctrl
{
 if (event.KeyCode == 28) // S
 {
  UICommand_SaveFile.Invoke();
 }
}
```

В результате по нажатию сочетания клавиш **«Ctrl»+«S»** будет активирована функция Invoke() экземпляра типа **Пользовательская команда**. Эта функция активирует обработчик события Invoked экземпляра типа **Пользовательская команда**, который сохранит содержимое экземпляра типа **Текст** в файл.

# 2.1.9.1. Свойства

## Включено (Enabled)

Включает или отключает пользовательскую команду. Тип значения - bool.

Значения:

- «`true`» - пользовательская команда включена и может выполняться;
- «`false`» - пользовательская команда отключена и не может выполняться.

### Триггер (InvokeTrigger)

Свойство-триггер пользовательской команды. Тип значения - bool.

Значение по умолчанию - «`false`», при записи значения «`true`» команда выполняется.

### Описание (Description)

Пользовательское описание команды. Тип значения - уведомляющий string.

### Сочетание клавиш (KeyGesture)

Описание сочетания клавиш, по нажатию которых активируется пользовательская команда. Тип значения - уведомляющий string.

ПРИМЕЧАНИЕ

Выполнение самой команды по нажатию "горячих" клавиш необходимо настроить в событии KeyPress формы. Подробнее об этом событии можно прочитать в Справочном руководстве на Alpha.HMI → Экранные формы → События → KeyPress.

## 2.1.9.2. События

### Invoked

Активация свойства Триггер или вызова функции Invoke().

### EnabledChanged

Изменение свойства Включено.

## 2.1.9.3. Функции

### Invoke

```
void Invoke()
```

Запускает пользовательскую команду, указанную в обработчике события Invoked.

## 2.1.10. Текстовые шаблоны

Вспомогательный тип, представляющий собой набор текстовых шаблонов, которые могут быть подставлены, например, в текст колонтитула в 2.2.8. Редактор колонтитула (стр. 120).

Принцип использования шаблонов в текстах колонтитулов приведен в описании компонента 2.2.8. Редактор колонтитула (стр. 120).

ОБРАТИТЕ ВНИМАНИЕ

Для использования шаблона в тексте нужно указать имя шаблона, обрамленное в специальные символы ${TemplateName}, где TemplateName - это имя шаблона. Пример приведен в описании функции Expand() компонента (стр. 44).

## 2.1.10.1. Свойства

### pAdditionData

Ссылка на таблицу, содержащую дополнительные текстовые шаблоны. Используется для объединения двух наборов шаблонов.

Дополнительные шаблоны могут быть добавлены из другого экземпляра компонента **Текстовые шаблоны**. Для этого в качестве значения свойства основного экземпляра укажите **«Template.Data»**, где **«Template»** - имя другого экземпляра компонента **Текстовые шаблоны**, **«Data»** - обращение к данным таблицы (шаблонам) этого экземпляра.

### pValueChanged

Ссылка на команду, выполняемую при изменении значения любого из шаблонов.

## 2.1.10.2. Функции

### Set

```
void Set(string name, variant value)
```

Устанавливает для шаблона name значение value. Имя шаблона указывается произвольно. Если значение указанного шаблона уже было задано ранее, оно будет обновлено.

ПРИМЕР

Примеры создания шаблонов:

```
Templates.Set("Department","Участок 1");
```

```
Templates.Set("CurrentUser", CurrentUserDisplayName);
```

```
Templates.Set("Now",DateTime.ToString(DateTime.Now()));
```

## Clear

```
void Clear()
```

Очищает набор шаблонов.

## Expand

```
string Expand(string text)
```

Возвращает текст, в котором названия шаблонов заменены их значениями. Шаблон следует вызывать, используя специальные символы: ${TemplateName}, где TemplateName - это имя шаблона. Если указать не существующий или не имеющий значения шаблон, то результирующий текст будет содержать обозначение шаблона без изменений.

ПРИМЕР

Пусть с помощью функции **Set()** в набор шаблонов *Templates* помещен шаблон *Department*, содержащий название участка производства.

```
Templates.Set("Department","Участок 1");
```

Предположим, в заголовке *Header* необходимо указать название участка из шаблона. Тогда вызов шаблона во входном параметре функции **Expand()** выглядит следующий образом:

```
Header.Text=Templates.Expand("Указаны объекты для ${Department}");
```

В результате в заголовок *Header* будет записан текст "Указаны объекты для Участок 1".

## ToJSON

```
string ToJSON()
```

Предоставляет перечень шаблонов, добавленных в текущий набор, в формате JSON.

Может использоваться для сохранения текущего набора в файл, создаваемый для последующей загрузки в экземпляр компонента.

Функция не учитывает дополнительные текстовые шаблоны, которые могут быть указаны в свойстве pAdditionData (стр. 43). Чтобы получить полный набор шаблонов, используйте функцию GetTemplateList (стр. 45).

ПРИМЕР

Пример значения, возвращаемого функцией:

```
[{"name":"Now","value":"30.11.2023 15:20:39"},
{"name":"CurrentUser","value":"Иванов"},
{"name":"Department","value":"Участок 1"}]
```

## FromJSON

```
void FromJSON(string json)
```

Разбирает указанный в формате JSON набор шаблонов. Используется для загрузки шаблонов в экземпляр компонента.

## GetTemplateList

```
string GetTemplateList()
```

Предоставляет перечень шаблонов, добавленных в текущий набор, в формате JSON, включая дополнительные наборы, указанные в свойстве pAdditionData (стр. 43). Чтобы получить набор шаблонов компонента без дополнительных наборов, используйте функцию ToJSON (стр. 44).

ПРИМЕР

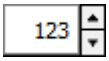
Пример значения, возвращаемого функцией:

```
[{"name":"ExtraTemplate","value":"Дополнительная информация"},
{"name":"Now","value":"30.11.2023 15:20:39"},
{"name":"CurrentUser","value":"Иванов"},
{"name":"Department","value":"Участок 1"}]
```

# 2.1.11. Редакторы

## 2.1.11.1. Редактор целого числа

Компонент, позволяющий менять значения целочисленных переменных с указанным шагом.



# Свойства

## Локализованный (Localized)

Позволяет указать, должны ли числа в редакторе отображаться с учетом знака разделителя групп разрядов, указанного в региональных настройках. Тип значения - bool.

Значение:

- **«true»** - число отображается с учетом знака разделителя: 12 566
- **«false»** - число отображается без учета знака разделителя: 12566

## Значение (Value)

Изменяемое значение. Тип значения - уведомляющий int8.

## Максимум (Maximum)

Максимум, до которого можно инкрементировать изменяемое значение. Тип значения - int8.

## Минимум (Minimum)

Минимум, до которого можно декрементировать изменяемое значение. Тип значения - int8.

## Шаг изменения (Step)

Шаг инкремента / декремента. Тип значения - int8.

## Событие изменения значения (ValueChanged)

Ссылка на команду, выполняемую при изменении значения.

## Событие изменения фокуса (FocusChanged)

Ссылка на команду, которая выполняется при получении фокуса редактора целого числа.

## В фокусе ввода (IsFocused)

Проверяет наличие или отсутствие фокуса у редактора. Тип значения - уведомляющий bool.

Значение:

- **«true»** - редактор в фокусе;
- **«false»** - редактор вне фокуса.

### Событие нажатия клавиши (pKeyPress)

Ссылка на обработчик события, который выполняется при нажатии клавиши в редакторе целого числа.

В обработчике события можно использовать аргументы возникшего события. Эти аргументы хранятся в виде JSON-строки в Буфере аргументов события (стр. 34). Получить их можно в обработчике события с помощью функции Args.Get().

ПРИМЕР

Пример JSON-строки, содержащей аргументы события, возникшего по нажатию клавиши **«5»** в редакторе целого числа

```
{
 "keyCode": 5,
 "modifiers": 0,
 "repeated": false,
 "text": "5"
}
```

Здесь:

- `keyCode` - код нажатой клавиши. Полный список кодов клавиш приведён в Справочном руководстве на Alpha.HMI → Компоненты окружения → Оборудование: клавиатура → Функции → подраздел Коды клавиш.
- `modifiers` - код клавиш-модификаторов (например, **«Ctrl»**, **«Shift»**, **«Alt»**), которые были зажаты в момент события. Если нажаты несколько таких клавиш, их значения суммируются. Полный список кодов модификаторов приведён в Справочном руководстве на Alpha.HMI → Компоненты окружения → Оборудование: клавиатура → Функции → подраздел Комбинации модификаторов.
- `repeated` - флаг удержания клавиши или сочетания любых клавиш. Не становится **«true»** при удержании клавиш-модификаторов: **«Ctrl»**, **«Shift»**, **«Alt»**.
- `text` - текст клавиши.

### Функции

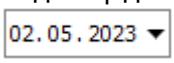
### SetFocus

```
void SetFocus()
```

Перевод фокуса на редактор. Функция не имеет входных параметров.

## 2.1.11.2. Редактор даты

Редактор даты для ввода даты в виде **«день:месяц:год»**.




Значения полей редактора можно менять:

- с помощью ввода цифр;
- клавишами-стрелками;

- выбрав дату из выпадающего календаря (стр. 110).

ПРИМЕЧАНИЕ

Можно использовать совместно с редактором времени (стр. 50).

## Свойства

### Значение (Value)

Хранит введенные в редакторе дату и время, округленное до секунд. Тип значения - уведомляющий timestamp.

При редактировании меняет значение даты, а значение времени оставляет без изменений.

### Событие изменения значения (pValueChanged)

Ссылка на команду, выполняемую при изменении даты в редакторе.

ПРИМЕР

При вводе даты в редакторе проверяется условие: если введенная дата больше текущей, появляется сообщение "Этот день еще не настал", в противном случае - "Дата выбрана". Тогда код обработчика команды:

```
DateEditor.Error = DateEditor.Value > DateTime.Now() ? "Этот день еще не
настал" : "Дата выбрана";
```

### Описание ошибки (Error)

Текст ошибки ввода даты. Тип значения - уведомляющий string.

ПРИМЕЧАНИЕ

Свойство не предназначено для проверки введенной даты. Проверку даты можно описать в команде, на которую ссылается свойство **pValueChanged**.

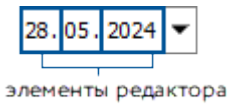
### Событие получения фокуса (pGotFocus)

Ссылка на команду, которая выполняется при получении фокуса у редактора даты.

### Номер поля с фокусом (LastFocused)

Возвращает порядковый номер элемента редактора, на котором был установлен фокус в последний раз. Тип значения - уведомляющий int4.

Нумерация элементов начинается с **«1»**.




## Событие нажатия клавиши (pKeyPress)

Ссылка на обработчик события, который выполняется при нажатии клавиши в редакторе даты.

В обработчике события можно использовать аргументы возникшего события. Эти аргументы хранятся в виде JSON-строки в Буфере аргументов события (стр. 34). Получить их можно в обработчике события с помощью функции Args.Get().

ПРИМЕР

Пример JSON-строки, содержащей аргументы события, возникшего по нажатию клавиши **«5»** в редакторе даты

```
{
 "keyCode": 5,
 "modifiers": 0,
 "repeated": false,
 "text": "5"
}
```

Здесь:

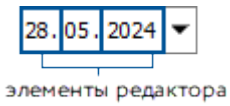
- `keyCode` - код нажатой клавиши. Полный список кодов клавиш приведён в Справочном руководстве на Alpha.HMI → Компоненты окружения → Оборудование: клавиатура → Функции → подраздел Коды клавиш.
- `modifiers` - код клавиш-модификаторов (например, **«Ctrl»**, **«Shift»**, **«Alt»**), которые были зажаты в момент события. Если нажаты несколько таких клавиш, их значения суммируются. Полный список кодов модификаторов приведён в Справочном руководстве на Alpha.HMI → Компоненты окружения → Оборудование: клавиатура → Функции → подраздел Комбинации модификаторов.
- `repeated` - флаг удержания клавиши или сочетания любых клавиш. Не становится **«true»** при удержании клавиш-модификаторов: **«Ctrl»**, **«Shift»**, **«Alt»**.
- `text` - текст клавиши.

## Функции

### SetFocus

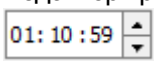
```
void SetFocus(int4 selectField)
```

Перевод фокуса на указанный элемента редактора. Входной параметр - порядковый номер элемента редактора. Нумерация элементов начинается с **«1»**.



## 2.1.11.3. Редактор времени

Редактор времени для ввода времени в виде **«часы:минуты:секунды»**.




Значения полей редактора можно менять:

- с помощью ввода цифр;
- клавишами-стрелками;
- кнопками-стрелками в редакторе.

ПРИМЕЧАНИЕ

Можно использовать совместно с редактором даты (стр. 47).

## Свойства

### Значение (Value)

Хранит введенные в редакторе дату и время, округленное до секунд. Тип значения - уведомляющий timestamp.

При редактировании меняет значение времени, а значение даты оставляет без изменений.

### Событие изменения значения (pValueChanged)

Ссылка на команду, выполняемую при изменении времени в редакторе.

ПРИМЕР

При вводе времени в редакторе проверяется условие: если введенный час вне границ рабочего времени, появляется сообщение "Рабочее время вышло", в противном случае - "Время выбрано". Тогда код обработчика команды:

```
if (DateTime.Hour(TimeEditor.Value) < 9 || DateTime.Hour(TimeEditor.Value)
> 18)
 TimeEditor.Error = "Рабочее время вышло";
else TimeEditor.Error = "Время выбрано";
```

### Описание ошибки (Error)

Текст ошибки ввода времени. Тип значения - уведомляющий string.

ПРИМЕЧАНИЕ

Свойство не предназначено для проверки введенного времени. Проверку времени можно описать в команде, на которую ссылается свойство **pValueChanged**.

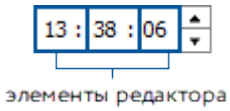
## Событие получения фокуса (pGotFocus)

Ссылка на команду, которая выполняется при получении фокуса редактора времени.

## Номер поля с фокусом (LastFocused)

Возвращает порядковый номер элемента редактора, на котором был установлен фокус в последний раз. Тип значения - уведомляющий int4.

Нумерация элементов начинается с **«1»**.




## Событие нажатия клавиши (pKeyPress)

Ссылка на обработчик события, который выполняется при нажатии клавиши в редакторе времени.

В обработчике события можно использовать аргументы возникшего события. Эти аргументы хранятся в виде JSON-строки в Буфере аргументов события (стр. 34). Получить их можно в обработчике события с помощью функции **Args.Get()**.

ПРИМЕР

Пример JSON-строки, содержащей аргументы события, возникшего по нажатию клавиши **«5»** в редакторе времени

```
{
 "keyCode": 5,
 "modifiers": 0,
 "repeated": false,
 "text": "5"
}
```

Здесь:

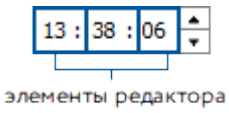
- `keyCode` - код нажатой клавиши. Полный список кодов клавиш приведён в Справочном руководстве на Alpha.HMI → Компоненты окружения → Оборудование: клавиатура → Функции → подраздел Коды клавиш.
- `modifiers` - код клавиш-модификаторов (например, **«Ctrl»**, **«Shift»**, **«Alt»**), которые были зажаты в момент события. Если нажаты несколько таких клавиш, их значения суммируются. Полный список кодов модификаторов приведён в Справочном руководстве на Alpha.HMI → Компоненты окружения → Оборудование: клавиатура → Функции → подраздел Комбинации модификаторов.
- `repeated` - флаг удержания клавиши или сочетания любых клавиш. Не становится **«true»** при удержании клавиш-модификаторов: **«Ctrl»**, **«Shift»**, **«Alt»**.
- `text` - текст клавиши.

## Функции

### SetFocus

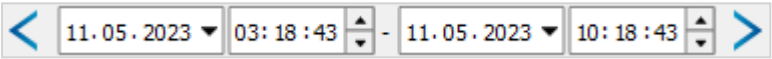
```
void SetFocus(int4 selectField)
```

Перевод фокуса на указанный элемента редактора. Входной параметр - порядковый номер элемента редактора. Нумерация элементов начинается с **«1»**.




## 2.1.11.4. Редактор интервала времени

Редактор интервала времени.




Позволяет указать интервал времени с точностью до секунд.
Значения полей редактора можно менять:

- с помощью ввода цифр;
- клавишами-стрелками;
- выбрав дату из выпадающего календаря (стр. 110).

Также можно использовать кнопки смещения к предыдущему и следующему интервалу времени.

## Свойства

### Begin

Хранит значение, соответствующее началу интервала, указанному в редакторе. Тип значения - уведомляющий timestamp.

### End

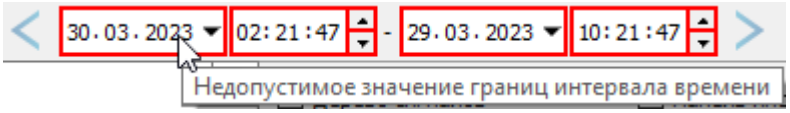
Хранит значение, соответствующее концу интервала, указанному в редакторе. Тип значения - уведомляющий timestamp.

### Error

Текст ошибки, возникшей при указании границ интервала. Тип значения - уведомляющий string.

ПРИМЕР

Если указать в качестве начала интервала значение, большее или равное значению окончания интервала, возникнет ошибка.




Текст ошибки записывается в компонент в качестве значения описываемого свойства.

## pIntervalChanged

Ссылка на команду, выполняемую при изменении границ интервала.

## pShiftCompleted

Ссылка на команду, выполняемую только при изменении границ интервала с помощью кнопок **Перейти к предыдущему интервалу времени**, **Перейти к следующему интервалу времени**.

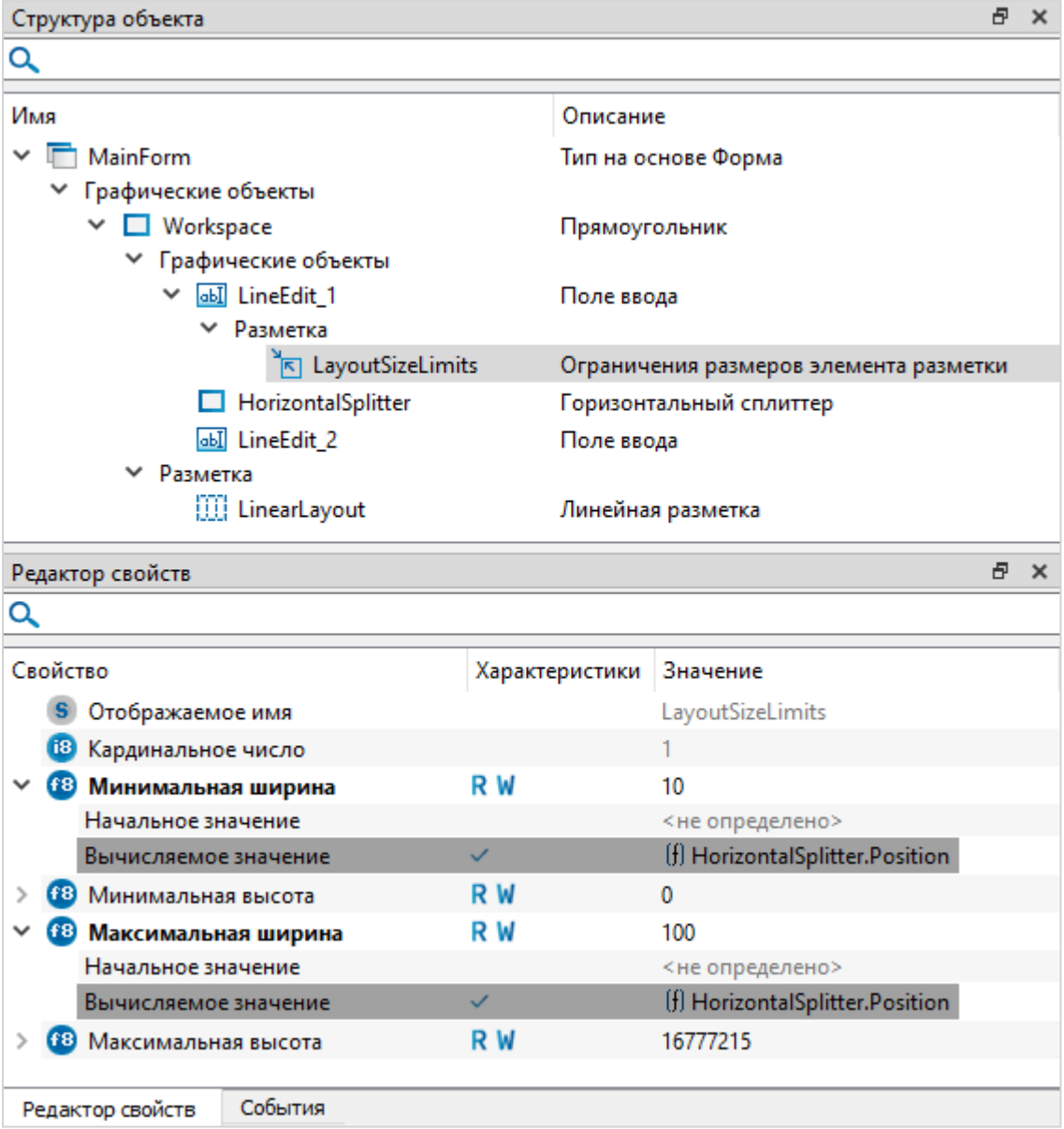
# 2.1.11.5. Горизонтальный сплиттер

Компонент, предназначенный для разделения области пользовательского интерфейса на левую и правую части с возможностью изменять размеры этих частей с помощью мышки.

Пример построения подобного визуального объекта описан ниже.

1. Поместите объекты, размеры которых должны меняться, в общий "контейнер". В примере на картинке роль таких объектов играют экземпляры типа **Поле ввода**: *Область 1* и *Область 2*, а роль "контейнера" - экземпляр типа **Прямоугольник**. Примените к "контейнеру" инструмент **Линейная разметка** с горизонтальной ориентацией.
2. Добавьте в "контейнер" экземпляр типа **Горизонтальный сплиттер**. Укажите его толщину, начальное положение, направление (в данном случае - **«Слева»**) и границы перемещения - минимум и максимум (подробнее - в описании свойств компонента). Обратите внимание, что в списке **Структура объекта** сплиттер должен находиться между объектами, меняющими размер.

3. К объекту, расположенному слева (*Область 1*), примените инструмент **Ограничения размеров элемента разметки**. Укажите в нем в качестве вычисляемых значений минимальной и максимальной ширины положение горизонтального сплиттера.




ПРИМЕЧАНИЕ

Применение горизонтального сплиттера невозможно без экземпляров типов **Линейная разметка** и **Ограничения размеров элемента разметки**.

## Свойства

### Толщина (Thickness)

Толщина разделителя. Тип значения - double.

### Положение (Position)

Положение разделителя относительно левого края "контейнера" по умолчанию. Тип значения - уведомляющий double.

### Минимум (Minimum)

Левый ограничитель перемещения сплиттера (относительно левого края "контейнера"). Тип значения - double.

### Максимум (Maximum)

Правый ограничитель перемещения сплиттера (относительно левого края "контейнера"). Тип значения - double.

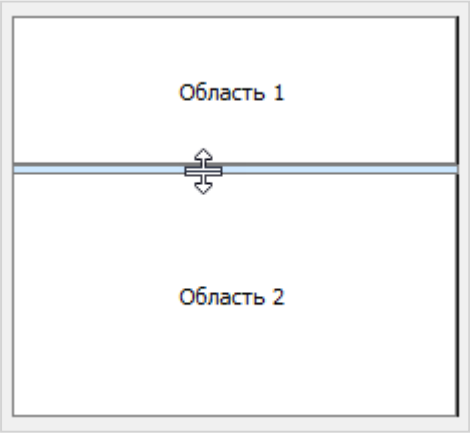
### Направление (HorizontalAlignment)

Определяет область "контейнера", ширину которой будет изменять разделитель при перемещении. Тип значения - int4.

Направление "По центру" не используется.

## 2.1.11.6. Вертикальный сплиттер

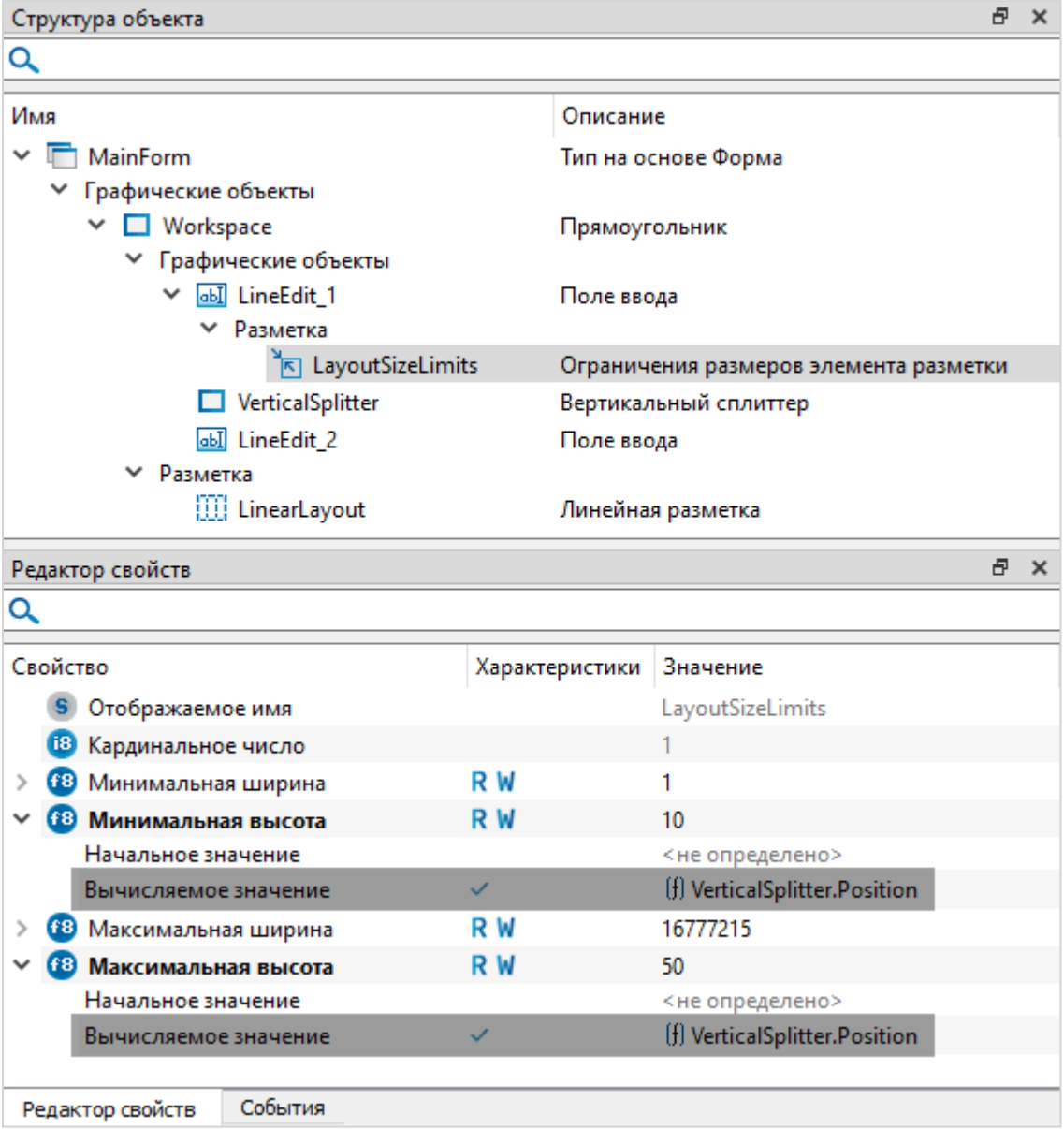
Компонент, предназначенный для разделения области пользовательского интерфейса на верхнюю и нижнюю части с возможностью изменять размеры этих частей с помощью мышки.




Пример построения подобного визуального объекта описан ниже.

1. Поместите объекты, размеры которых должны меняться, в общий "контейнер". В примере на картинке роль таких объектов играют экземпляры типа **Поле ввода**: *Область 1* и *Область 2*, а роль "контейнера" - экземпляр типа **Прямоугольник**. Примените к "контейнеру" инструмент **Линейная разметка** с вертикальной ориентацией.
2. Добавьте в "контейнер" экземпляр типа **Вертикальный сплиттер**. Укажите его толщину, начальное положение, направление (в данном случае - **«Сверху»**) и границы перемещения - минимум и максимум (подробнее - в описании свойств компонента). Обратите внимание, что в списке **Структура объекта** сплиттер должен находиться между объектами, меняющими размер.

3. К объекту, расположенному сверху (*Область 1*), примените инструмент **Ограничения размеров элемента разметки**. Укажите в нем в качестве вычисляемых значений минимальной и максимальной высоты положение вертикального сплиттера.




ПРИМЕЧАНИЕ

Применение вертикального сплиттера невозможно без экземпляров типов **Линейная разметка** и **Ограничения размеров элемента разметки**.

## Свойства

### Толщина (Thickness)

Толщина разделителя. Тип значения - double.

### Положение (Position)

Положение разделителя относительно верхнего края "контейнера" по умолчанию. Тип значения - уведомляющий double.

### Минимум (Minimum)

Верхний ограничитель перемещения сплиттера (относительно верхнего края "контейнера"). Тип значения - double.

### Максимум (Maximum)

Нижний ограничитель перемещения сплиттера (относительно верхнего края "контейнера"). Тип значения - double.

### Направление (HorizontalAlignment)

Определяет область "контейнера", ширину которой будет изменять разделитель при перемещении. Тип значения - int4.

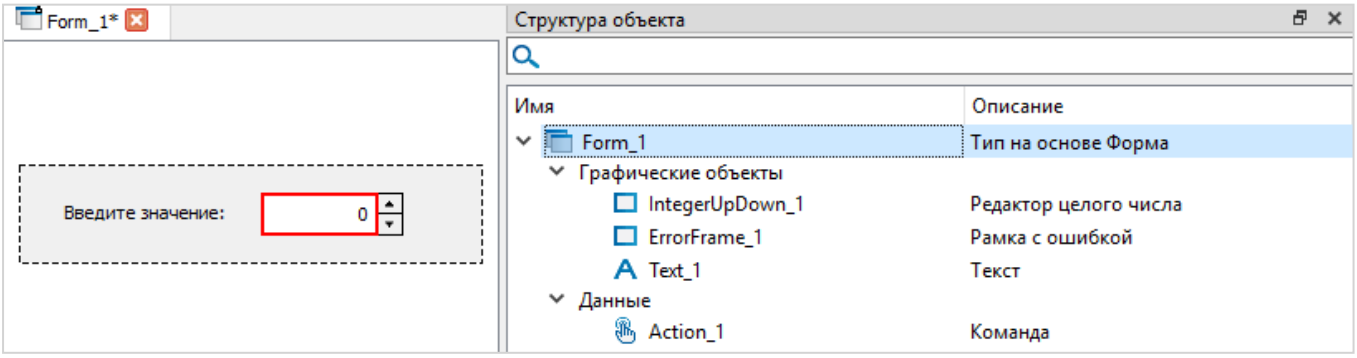
Направление "По центру" не используется.

## 2.1.11.7. Рамка с ошибкой

Вспомогательный тип, используемый для индикации ошибки. При возникновении ошибки элемент подсвечивается красной рамкой, отображается всплывающая подсказка.

Пусть требуется выводить сообщение об ошибке при вводе отрицательного значения в экземпляре типа 2.1.11.1. Редактор целого числа (стр. 45). Для этого добавьте в проект экземпляры типов:

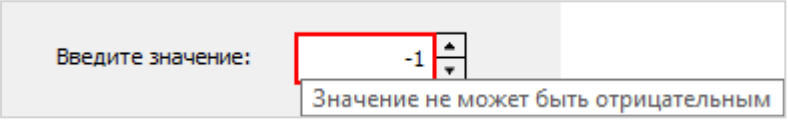
- 2.1.11.1. Редактор целого числа (стр. 45). Здесь будет выполняться ввод значения.
- **Рамка с ошибкой**. Эта рамка и всплывающая подсказка будут появляться при вводе отрицательного значения. Расположите экземпляр этого типа так, чтобы он подсвечивал необходимое поле.
- **Команда**. Команда, которая будет выполнять проверку значения целого числа при его изменении в экземпляре типа 2.1.11.1. Редактор целого числа (стр. 45).



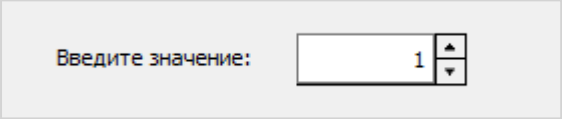

Чтобы при изменении значения в экземпляре типа 2.1.11.1. Редактор целого числа (стр. 45) происходила проверка этого значения, в свойстве **ValueChanged** этого экземпляра сошлитесь на созданную команду. В событии **Invoked** этой команды можно использовать следующий код, написанный на языке Alpha.Om.

```
if(IntegerUpDown_1.Value<0)
{
 ErrorFrame_1.Error="Значение не может быть отрицательным";
}
else
{
 ErrorFrame_1.Error="";
}
```

В результате при вводе отрицательных значений будет подсвечиваться поле ввода и выводиться сообщение об ошибке.




При появлении положительных значений рамка и сообщение об ошибке будут исчезать.




## Свойства

### Описание ошибки (Error)

Описание ошибки, которое отображается в виде всплывающей подсказки. Тип значения - уведомляющий string.

Если значение этого свойства не пустое, то экземпляр типа **Рамка с ошибкой** становится видимым.

### Интервал до открытия, мс (InitialShowDelay)

Время до отображения всплывающей подсказки. Тип значения - уведомляющий int4.

### Время отображения, мс (ShowDuration)

Продолжительность отображения всплывающей подсказки. Тип значения - уведомляющий int4.

## 2.1.11.8. Редактор даты и времени

Редактор ввода даты и времени в виде **«день:месяц:год»** и **«часы:минуты:секунды»**.

Значения полей редактора можно менять:

- с помощью ввода цифр;
- клавишами-стрелками.

Значение поля даты можно выбирать из выпадающего календаря (стр. 110).

Значение поля времени можно выбирать кнопками-стрелками в редакторе.

## Свойства

### Значение (Value)

Хранит введенные в редакторе дату и время, округленное до секунд. Тип значения - уведомляющий timestamp.

При редактировании меняет значение даты и времени.

### Событие изменения значения (pValueChanged)

Ссылка на команду, выполняемую при изменении значения в редакторе.

### Описание ошибки (Error)

Текст ошибки ввода даты. Тип значения - уведомляющий string.

ПРИМЕЧАНИЕ

Свойство не предназначено для проверки введенной даты. Проверку даты можно описать в команде, на которую ссылается свойство pValueChanged.

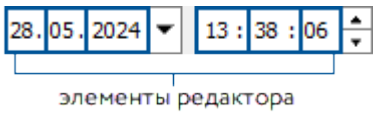
### Событие получения фокуса (pGotFocus)

Ссылка на команду, которая выполняется при получении фокуса редактора даты и времени.

### Номер поля с фокусом (LastFocused)

Возвращает порядковый номер элемента редактора, на котором был установлен фокус в последний раз. Тип значения - уведомляющий int4.

Нумерация элементов начинается с **«1»**.




### Событие нажатия клавиши (pKeyPress)

Ссылка на обработчик события, который выполняется при нажатии клавиши в редакторе даты и времени.

В обработчике события можно использовать аргументы возникшего события. Эти аргументы хранятся в виде JSON-строки в Буфере аргументов события (стр. 34). Получить их можно в обработчике события с помощью функции **Args.Get()**.

ПРИМЕР

Пример JSON-строки, содержащей аргументы события, возникшего по нажатию клавиши **«5»** в редакторе даты и времени

```
{
 "keyCode": 5,
 "modifiers": 0,
 "repeated": false,
 "text": "5"
}
```

Здесь:

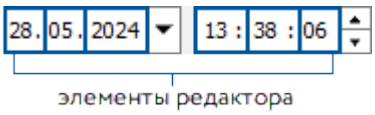
- `keyCode` - код нажатой клавиши. Полный список кодов клавиш приведён в Справочном руководстве на Alpha.HMI → Компоненты окружения → Оборудование: клавиатура → Функции → подраздел Коды клавиш.
- `modifiers` - код клавиш-модификаторов (например, **«Ctrl»**, **«Shift»**, **«Alt»**), которые были зажаты в момент события. Если нажаты несколько таких клавиш, их значения суммируются. Полный список кодов модификаторов приведён в Справочном руководстве на Alpha.HMI → Компоненты окружения → Оборудование: клавиатура → Функции → подраздел Комбинации модификаторов.
- `repeated` - флаг удержания клавиши или сочетания любых клавиш. Не становится **«true»** при удержании клавиш-модификаторов: **«Ctrl»**, **«Shift»**, **«Alt»**.
- `text` - текст клавиши.

## Функции

### SetFocus

```
void SetFocus(int4 selectField)
```

Перевод фокуса на указанный элемента редактора. Входной параметр - порядковый номер элемента редактора. Нумерация элементов начинается с **«1»**.




## 2.1.12. Меню

### 2.1.12.1. Элемент меню

Элемент контекстного 2.2.2. Меню (стр. 107). Используется в меню, но также может использоваться как отдельный компонент.

ПРИМЕЧАНИЕ

После изменения размеров компонента для обновления разметки необходимо вызвать метод **UpdateLayout(keyGestureWidth: double)**. Входной параметр необходимо указать для того, чтобы ширина правой части с сочетаниями клавиш была одинаковой для всех элементов меню.

## Свойства

### IconFile

Добавляет элементу меню иконку. В качестве значения укажите путь к файлу иконки. Тип значения - уведомляющий string.

ПРИМЕР

"Files_Icons/SaveToFile16.png"

### Title

Описание команды меню. Тип значения - уведомляющий string.

### KeyGesture

Значение, отображаемое в правой части элемента меню. Тип значения - уведомляющий string.

### Checked

Преобразует элемент меню в элемент выбора с иконкой ✓. Тип значения - уведомляющий bool.

Значение:

- «`true`» - элемент выбора;
- «`false`» - обычный элемент меню.

### Selected

Указывает, выбран (выделен) ли элемент меню. Тип значения - уведомляющий bool.

Значение:

- «`true`» - элемент выбран;
- «`false`» - элемент не выбран.

### Active

Активность элемента меню. Тип значения - уведомляющий bool.

Значение:

- «`true`» - элемент активен;
- «`false`» - элемент неактивен.

### Items

Дочерние элементы меню. Тип значения - уведомляющий string.

## 2.1.12.2. Разделитель элементов меню

Разделитель элементов 2.2.2. Меню (стр. 107). Используется в меню, но также может использоваться как отдельный компонент.

Собственных свойств не имеет, так как представляет собой горизонтальную линию.

> ПРИМЕЧАНИЕ
> После изменения размеров компонента для обновления разметки необходимо вызвать метод **UpdateLayout()**.

## 2.1.12.3. Результат выбора меню

Компонент, в котором сохраняется результат выбора пункта в 2.2.2. Меню (стр. 107).

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

### Свойства

### Идентификатор команды (Id)

Идентификатор выбранного (выделенного) пункта меню. Тип значения - string.

### Меню остается открытым (StaysOpen)

Указывает, остается ли открытым меню при выборе (выделении) этого пункта меню. Тип значения - bool.

### Обновить элементы меню (UpdateItems)

Позволяет обновлять элементы меню после его открытия. Значение данного свойства представляет собой json-строку. Тип значения - уведомляющий string. Подробнее структура json-строки описана в свойстве Items компонента Menu (стр. 108). Строка-значение данного свойства должна описывать измененное состояние пунктов меню.

Используется, например, если выбор одного пункта меню должен приводить к изменению других пунктов меню: их заголовков, иконок, доступности.

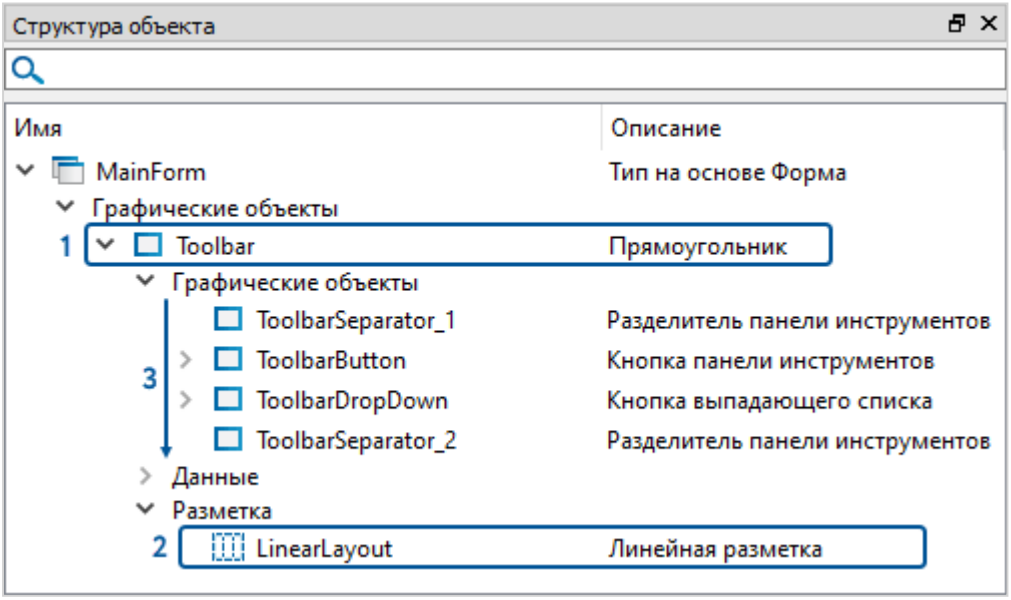
ПРИМЕР

```
MenuResult.UpdateItems = JSON.stringify([
 {
 'id': "addSignal",
 'title': "Добавить новый сигнал...",
 'active': true
 }
 {
 'id': "saveSignal",
 'title': "Сохранить сигнал...",
 'active': true
 }
]);
```

## 2.1.13. Панель инструментов

Панель инструментов в проекте Alpha.HMI можно построить из экземпляров компонентов 2.1.13.1. Кнопка панели инструментов (стр. 64), 2.1.13.2. Кнопка выпадающего списка (стр. 65), 2.1.13.4. Панель поиска (стр. 67) и 2.1.13.3. Разделитель панели инструментов (стр. 67). Чтобы создать панель инструментов:

1. Создайте графический объект типа **Прямоугольник**.
2. Примените к нему инструмент **Линейная разметка**. Это необходимо для корректного поведения компонентов панели.
3. Экземпляры компонентов панели размещайте внутри созданного прямоугольника. Учитывайте порядок отображения экземпляров в структуре объекта: в таком же порядке они будут расположены на панели инструментов.




Пример построения панели инструментов приведен в 1.4.1. Выбор цвета на панели инструментов из палитры или из меню (стр. 10).

## 2.1.13.1. Кнопка панели инструментов

Кнопка для панели инструментов.

Внешний вид и функции кнопки можно менять с помощью свойств и событий. Кнопку можно использовать совместно с 2.1.13.2. Кнопка выпадающего списка (стр. 65). Для визуального разделения панели инструментов можно использовать 2.1.13.3. Разделитель панели инструментов (стр. 67).

Для корректного поведения размещайте экземпляр компонента внутри 2.1.13. Панель инструментов (стр. 63).

Пример использования приведен в 1.4.1. Выбор цвета на панели инструментов из палитры или из меню (стр. 10).

## Свойства

### IconFile

Путь к файлу иконки для текущего экземпляра кнопки. Тип значения - уведомляющий string.

### TitleText

Текст кнопки. Тип значения - уведомляющий string.

Для корректного отображения текста необходимо вручную подобрать ширину кнопки с помощью свойства ButtonWidth (стр. 65).

### Checked

Позволяет использовать кнопку как удерживаемую. Тип значения - уведомляющий bool.

Значение:

- **«true»** - кнопка удерживаемая, выглядит как нажатая.
- **«false»** - кнопка отпущена, выглядит как обычно.

### Active

Позволяет изменить активность кнопки. Тип значения - уведомляющий bool.

Значение:

- **«true»** - кнопка активна и выглядит как обычно, при наведении мышкой подсвечивается.
- **«false»** - кнопка неактивна: цвета иконки бледные, при наведении мышкой кнопка не подсвечивается.

### IsVisible

Позволяет изменить видимость кнопки. Тип значения - уведомляющий bool.

Значение:

- **«true»** - кнопка отображается и выглядит как обычно.
- **«false»** - кнопка не отображается, так как ее ширина становится нулевой, благодаря чему выравниваются остальные элементы панели.

### pDropDown

Ссылка на определенный экземпляр 2.1.13.2. Кнопка выпадающего списка (стр. 65).

Если указать значение, то в рантайме при наведении мышкой на текущий экземпляр **ToolbarButton** подсвечиваться также будет указанный экземпляр **ToolbarDropDown**.

### Margin

Устанавливает размер отступов для расположения иконки относительно кнопки. Тип значения - uint1.

### IconSize

Размер иконки для текущего экземпляра кнопки. Тип значения - uint1.

### ButtonWidth

Ширина кнопки. Тип значения - уведомляющий uint2.

### ButtonHeight

Высота кнопки. Тип значения - уведомляющий uint2.

## 2.1.13.2. Кнопка выпадающего списка

Кнопка открытия выпадающего элемента (2.2.2. Меню (стр. 107), 2.2.3. Календарь (стр. 110), и т.п.) для панели инструментов.

Внешний вид и функции кнопки можно менять с помощью свойств и событий. Кнопку можно использовать совместно с 2.1.13.1. Кнопка панели инструментов (стр. 64). Для визуального разделения панели инструментов можно использовать 2.1.13.3. Разделитель панели инструментов (стр. 67).

Для корректного поведения размещайте экземпляр компонента внутри 2.1.13. Панель инструментов (стр. 63). Пример использования приведен в 1.4.1. Выбор цвета на панели инструментов из палитры или из меню (стр. 10).

## Свойства

### Checked

Позволяет использовать кнопку как удерживаемую. Тип значения - уведомляющий bool.

Значение:

- **«true»** - кнопка удерживаемая, выглядит как нажатая.
- **«false»** - кнопка отпущена, выглядит как обычно.

### Active

Позволяет изменить активность кнопки. Тип значения - уведомляющий bool.

Значение:

- **«true»** - кнопка активна и выглядит как обычно, при наведении мышкой подсвечивается.
- **«false»** - кнопка неактивна: цвет стрелки светло-серый, при наведении мышкой кнопка не подсвечивается.

### IsVisible

Позволяет изменить видимость кнопки. Тип значения - уведомляющий bool.

Значение:

- **«true»** - кнопка отображается и выглядит как обычно.
- **«false»** - кнопка не отображается, так как ее ширина становится нулевой, благодаря чему выравниваются остальные элементы панели.

### pButton

Ссылка на определенный экземпляр 2.1.13.1. Кнопка панели инструментов (стр. 64).

Если указать значение, то в рантайме при наведении мышкой на текущий экземпляр **ToolbarDropDown** подсвечиваться также будет указанный экземпляр **ToolbarButton**.

### Margin

Устанавливает размер отступов для расположения иконки относительно кнопки. Тип значения - uint1.

### ButtonWidth

Ширина кнопки. Тип значения - уведомляющий uint2.

### ButtonHeight

Высота кнопки. Тип значения - уведомляющий uint2.

## 2.1.13.3. Разделитель панели инструментов

Разделитель элементов для панелей инструментов.

Для корректного поведения размещайте экземпляр компонента внутри 2.1.13. Панель инструментов (стр. 63).

Пример использования приведен в 1.4.1. Выбор цвета на панели инструментов из палитры или из меню (стр. 10).

## Свойства

### IsVisible

Позволяет изменить видимость разделителя. Тип значения - уведомляющий bool.

Значение:

- **«true»** - разделитель отображается.
- **«false»** - разделитель не отображается, так как его ширина становится нулевой, благодаря чему выравниваются остальные элементы панели.

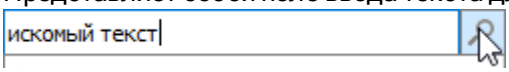
### ButtonHeight

Высота разделителя. Тип значения - уведомляющий uint2.

## 2.1.13.4. Панель поиска

Панель поиска.

Представляет собой поле ввода текста для поиска (шаблона) и кнопку запуска процесса поиска.




После ввода шаблона кнопка меняет свое назначение и позволяет очистить результаты поиска.

## Свойства

### Текст заполнителя (PlaceholderText)

Текст, отображаемый в поле ввода по умолчанию. Тип значения - уведомляющий string.

### IsVisible

Меняет видимость панели поиска, меняя ее ширину. Тип значения - уведомляющий bool.

Значение:

- **«true»** - панель видна, ширина панели обычная;
- **«false»** - панель скрывается, так как ее высота становится нулевой, благодаря чему выравниваются другие элементы.

### Шаблон поиска (Pattern)

Хранит введенный текст для поиска (шаблон). Тип значения - уведомляющий string.

- Изменяется при нажатии клавиши Enter или кнопки запуска процесса поиска.
- Сбрасывается при нажатии клавиши Esc или кнопки очистки результатов поиска.

### pPatternChanged

Ссылка на команду, выполняемую при изменении шаблона в Pattern.

### pGotFocus

Ссылка на команду, которая должна вызываться при получении фокуса у панели поиска.

## Функции

### SetFocus

```
void SetFocus()
```

Перевод фокуса на панель поиска.

## 2.1.13.5. Тема оформления панели инструментов

Компонент, позволяющий менять тему оформления визуальных компонентов Alpha.HMI.CommonLib, добавленных на панель инструментов. Свойства компонента, значения которых меняют внешний вид элементов панели инструментов, ссылаются на свойства основного элемента типа 2.1.1. Тема оформления (стр. 18) (см. Свойства (стр. 68)).

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

## Свойства

### Тема оформления (init_Theme)

Ссылка на экземпляр типа 2.1.1. Тема оформления (стр. 18), значения свойств которого должны быть применены на панели инструментов.

## Шрифт (Font)

Шрифт основного текста. Ссылается на значение аналогичного свойства экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - string.

## Шрифт заголовков (TitleFont)

Шрифт заголовков. Ссылается на значение аналогичного свойства экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - string.

## Цвет фона (BackgroundColor)

Цвет основной области компонента. Ссылается на значение свойства **Цвет фона панели инструментов** экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет текста (ForegroundColor)

Цвет текста в основной области компонента. Ссылается на значение свойства **Цвет текста на панели инструментов** экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет текста при выключении (DisabledFontColor)

Цвет текста неактивных элементов. Ссылается на значение аналогичного свойства экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет фона редактора (EditorBrushColor)

Цвет фона области компонента, в которой можно изменить значение. Ссылается на значение свойства **Цвет редактора на панели инструментов** экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет текста в редакторе (EditorFontColor)

Цвет текста в области компонента, в которой можно изменить значение. Ссылается на значение свойства **Цвет текста на панели инструментов** экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет выделения (SelectionColor)

Цвет активного (выбранного) элемента в области компонента, в которой можно изменить значение. Ссылается на значение свойства **Цвет выделения на панели инструментов** экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет рамки (BorderColor)

Цвет рамки, ограничивающей область компонента, в которой можно изменить значение. Ссылается на значение аналогичного свойства экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет рамки при выключении (DisabledBorderColor)

Цвет рамки, ограничивающей область компонента, в которой можно изменить значение, если эта область неактивна. Ссылается на значение аналогичного свойства экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет фона панели инструментов (ToolbarBrushColor)

Цвет фона панели инструментов. Ссылается на значение аналогичного свойства экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет текста на панели инструментов (ToolbarFontColor)

Цвет текста на панели инструментов. Ссылается на значение аналогичного свойства экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет редактора на панели инструментов (ToolbarEditorColor)

Цвет редакторы на панели инструментов. Ссылается на значение аналогичного свойства экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет выделения на панели инструментов (ToolbarSelectionColor)

Цвет выделения элемента на панели инструментов. Ссылается на значение аналогичного свойства экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет фона меню (MenuBrushColor)

Цвет фона меню. Ссылается на значение свойства **Цвет фона панели инструментов** экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет текста в меню (MenuFontColor)

Цвет текста в меню. Ссылается на значение свойства **Цвет текста на панели инструментов** экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## Цвет текста недоступного элемента меню (MenuDisabledFontColor)

Цвет текста недоступного элемента меню. Ссылается на значение свойства **Цвет текста при выключении** экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

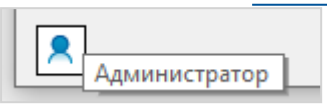
## Цвет выделенного элемента меню (MenuSelectionColor)

Цвет выделенного элемента меню. Ссылается на значение свойства **Цвет выделения на панели инструментов** экземпляра типа 2.1.1. Тема оформления (стр. 18). Тип значения - уведомляющий uint4.

## 2.1.14. Строка статуса

### 2.1.14.1. Кнопка авторизации

Кнопка вызова окна 2.2.10.1. Диалог авторизации пользователя (стр. 133).




Внешний вид кнопки зависит от текущего состояния входа:

- - пользователь авторизован в системе;
- - гостевой режим.

Во всплывающей подсказке показывается отображаемое имя текущего пользователя.

## Свойства

### Показывать диалог поверх всех окон (ShowDialogOnTop)

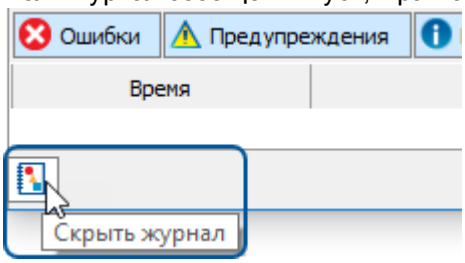
Позволяет настроить отображение открываемого по нажатию на кнопку окна 2.2.10.1. Диалог авторизации пользователя (стр. 133) поверх остальных окон. Тип значения - уведомляющий bool.

Значение:

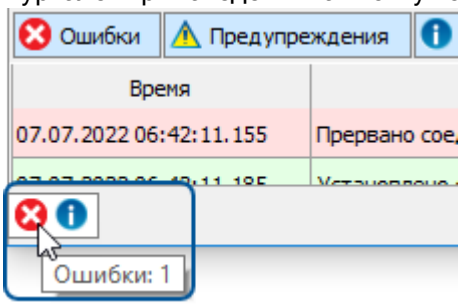
- «`true`» - отображать окно 2.2.10.1. Диалог авторизации пользователя (стр. 133) поверх остальных окон;
- «`false`» - не отображать окно 2.2.10.1. Диалог авторизации пользователя (стр. 133) поверх остальных окон.

### 2.1.14.2. Кнопка-индикатор журнала сообщений

Кнопка-индикатор наличия сообщений в журнале (стр. 88). Позволяет менять видимость журнала.
Если журнал сообщений пуст, в рантайме кнопка выглядит следующим образом:




Если в журнале есть сообщения, на кнопке появляются иконки, соответствующие типам сообщений в журнале. При наведении на иконку всплывает подсказка с количеством сообщений этого типа в журнале:



## Свойства

### pLogView

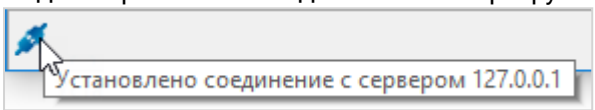
Ссылка на определенный экземпляр журнала сообщений (стр. 88).

### ButtonHeight

Высота кнопки. Тип значения - uint2.

## 2.1.14.3. Индикатор подключения к серверу

Индикатор состояния подключения к серверу.




Во всплывающей подсказке выводится информация о состоянии подключения.

## Свойства

### Ссылка на источник (init_ApSource)

Здесь указывается сервер, о состоянии подключения с которым должен сообщать **ConnectionIndicator**. Тип значения - ссылка на источник данных типа **Источник АР**.

### Описание состояния (StateInfo)

Содержит информацию о подключении в виде json-строки. Тип значения - уведомляющий string.

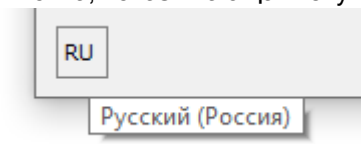
ПРИМЕР

```
{
"importance": "info",
"description": "Установлено соединение с сервером 127.0.0.1"
}
```

```
{
"importance": "error",
"description": "Прервано соединение с сервером 127.0.0.1"
}
```

## 2.1.14.4. Кнопка смены раскладки клавиатуры

Кнопка, показывающая текущую раскладку клавиатуры.




При нажатии на кнопку раскладка меняется.

ПРИМЕЧАНИЕ

Кнопка применяется в компонентах 2.2.10.1. Диалог авторизации пользователя (стр. 133) и 2.2.10.2. Диалог смены пароля (стр. 136).

## 2.1.14.5. Кнопка вызова экранной клавиатуры

Кнопка вызова экранной клавиатуры.

ПРИМЕЧАНИЕ

Используя компонент, создайте экземпляр типа 2.1.14.6. Параметры вызова экранной клавиатуры (стр. 73), в котором будут сохранены настройки вызова экранной клавиатуры. Ссылка на созданный экземпляр указывается в свойстве Параметры запуска (стр. 73).

Кнопка уже есть в:

- диалогах авторизации (2.2.10.1. Диалог авторизации пользователя (стр. 133), 2.2.10.2. Диалог смены пароля (стр. 136));
- диалоге подключения к серверу (2.2.7. Диалог подключения к серверу (стр. 118), 2.2.10.3. Диалог ввода учетных данных (стр. 137));
- диалогах ввода значения (2.2.5. Диалог ввода значения (стр. 115));
- диалогах открытия и сохранения файла (2.2.9.1. Диалог открытия файла (стр. 122), 2.2.9.2. Диалог сохранения файла (стр. 127)).

Отображение кнопки в указанных компонентах необходимо настроить.

## Свойства

### Параметры запуска (init_Options)

Ссылка на экземпляр типа 2.1.14.6. Параметры вызова экранной клавиатуры (стр. 73), в котором хранятся настройки вызова экранной клавиатуры.

## 2.1.14.6. Параметры вызова экранной клавиатуры

Компонент, в котором хранятся настройки вызова экранной клавиатуры.
Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

# Свойства

## Включено (Enabled)

Настройка видимости кнопки вызова экранной клавиатуры. Тип значения - уведомляющий bool.

Значение:

- **«true»** - кнопка отображается;
- **«false»** - кнопка скрыта.

## Путь к программе (ProgramPath)

Полный путь к программе экранной клавиатуры. Тип значения - уведомляющий string.

Можно не указывать. По умолчанию будет использоваться:

- в Windows - `alpha.hmi.keyboard.exe` (поставляется вместе с дистрибутивом Alpha.HMI).
- в Linux - `/usr/bin/fly-vkbd`.

## Аргументы запуска (Arguments)

Аргументы запуска программы экранной клавиатуры. Зависят от используемой программы. Тип значения - уведомляющий string.

Аргументы для программы `alpha.hmi.keyboard.exe`, поставляемой вместе с дистрибутивом Alpha.HMI для Windows, приведены в таблице ниже.

| Аргумент | Назначение |
|---|---|
| --window-pos-x VALUE, --window-pos-y VALUE | Управляет стартовой позицией окна, где VALUE - координата верхнего левого угла окна. |
| --window-height VALUE, --window-width VALUE | Управляет стартовыми размерами окна, где VALUE - высота/ширина окна. |
| --no-window-resize | Запрещает изменять размер окна экранной клавиатуры. |
| --no-window-minimize | Запрещает сворачивать окно экранной клавиатуры. |
| --no-window-maximize | Запрещает разворачивать окно экранной клавиатуры. |
| --no-window-close | Запрещает закрывать окно экранной клавиатуры. |

| Аргумент | Назначение |
|---|---|
| --use-security | Учитывает право доступа к настройкам экранной клавиатуры текущего пользователя подсистемы безопасности.<br>› Если в Alpha.Security пользователю назначено право **EditSettings (Изменение настроек)** со значением **«true»**, он сможет изменить настройки экранной клавиатуры по нажатию кнопки **Настройки**.<br>› Если право назначено со значением **«false»** или не назначено вовсе, кнопка **Настройки** для него будет заблокирована. |

## 2.1.15. Диалоги

### 2.1.15.1. Нажата клавиша Caps Lock

Вспомогательный компонент, предназначенный для предупреждения о нажатой клавише **Caps Lock** в диалогах с вводом пароля. При нажатой клавише **Caps Lock** этот компонент становится видимым.

### 2.1.15.2. Результат выбора папки

Компонент, в котором сохраняется путь к папке, выбранной в окне 2.2.9.3. Диалог выбора папки (стр. 132).

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

#### Свойства

##### Путь к папке (FolderPath)

Хранит путь к папке, выбранной в окне 2.2.9.3. Диалог выбора папки (стр. 132). Тип значение - уведомляющий string.

### 2.1.15.3. Результат выбора цвета

Компонент, в котором сохраняется значение, введенное в окне 2.2.6. Диалог выбора цвета (стр. 116).

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

#### Свойства

##### Цвет (Color)

Хранит цвет, выбранный в окне 2.2.6. Диалог выбора цвета (стр. 116). Тип значения - уведомляющий uint4.

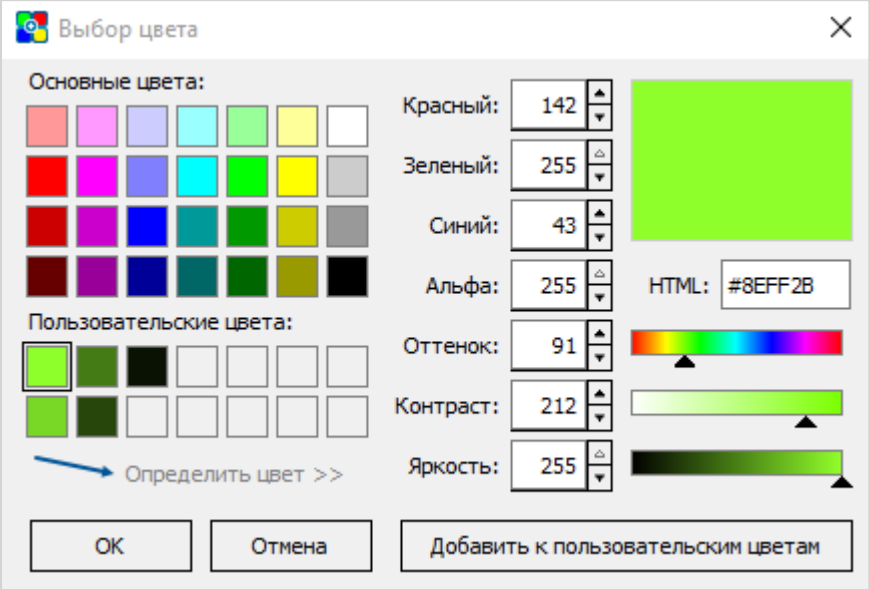
##### Пользовательская палитра (CustomColors)

Хранит перечень пользовательских цветов в виде массива чисел в формате JSON. Тип значения - string.

ПРИМЕР

```
[ "0xffffa500", "0xff9400d3" ]
```

Добавить цвет в список пользовательских можно, расширив диалоговое окно и нажав кнопку **Добавить к пользовательским цветам**.




Всего может быть указано до 14 цветов.

## 2.1.15.4. Результат выбора параметров подключения

Компонент, в котором сохраняются адрес и порт сервера, к которому выполняется подключение в окне 2.2.7. Диалог подключения к серверу (стр. 118).

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

## Свойства

### Название (Title)

Название подключения. Тип значения - string.

### Хост (Location)

IP-адрес или имя сервера (компьютера), к которому выполняется подключение. Тип значения - string.

### Порт (Port)

Номер порта, через который выполняется подключение к указанному серверу (компьютеру). Тип значения - uint2.

### Порт истории (HistoryPort)

Номер порта истории. Тип значения - uint2.

### Учетные данные (Credentials)

Учетные данные, указываемые при подключении к защищенному паролем серверу. Тип значения - string.

## 2.1.15.5. Результат выбора даты и времени

Компонент, в котором сохраняется дата, выбранная в 2.2.3. Календарь (стр. 110).

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

## Свойства

### Значение (Value)

Хранит дату, выбранную в 2.2.3. Календарь (стр. 110). Тип значения - timestamp.

## 2.1.15.6. Результат ввода значения

Компонент, в котором сохраняется значение, введенное в окне 2.2.5. Диалог ввода значения (стр. 115).

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

## Свойства

### Значение (Value)

Итоговое значение, введенное пользователем. Тип значения - уведомляющий string.

### Вводимый текст (InputText)

Вводимый текст. Тип значения - уведомляющий string.

Может использоваться:

- для проверки введенного текста;
- в качестве начального значения при открытии окна **InputDialog** (стр. 115).

### Описание ошибки (Error)

Описание ошибки ввода. Тип значения - уведомляющий string.

При возникновении ошибки ввода:

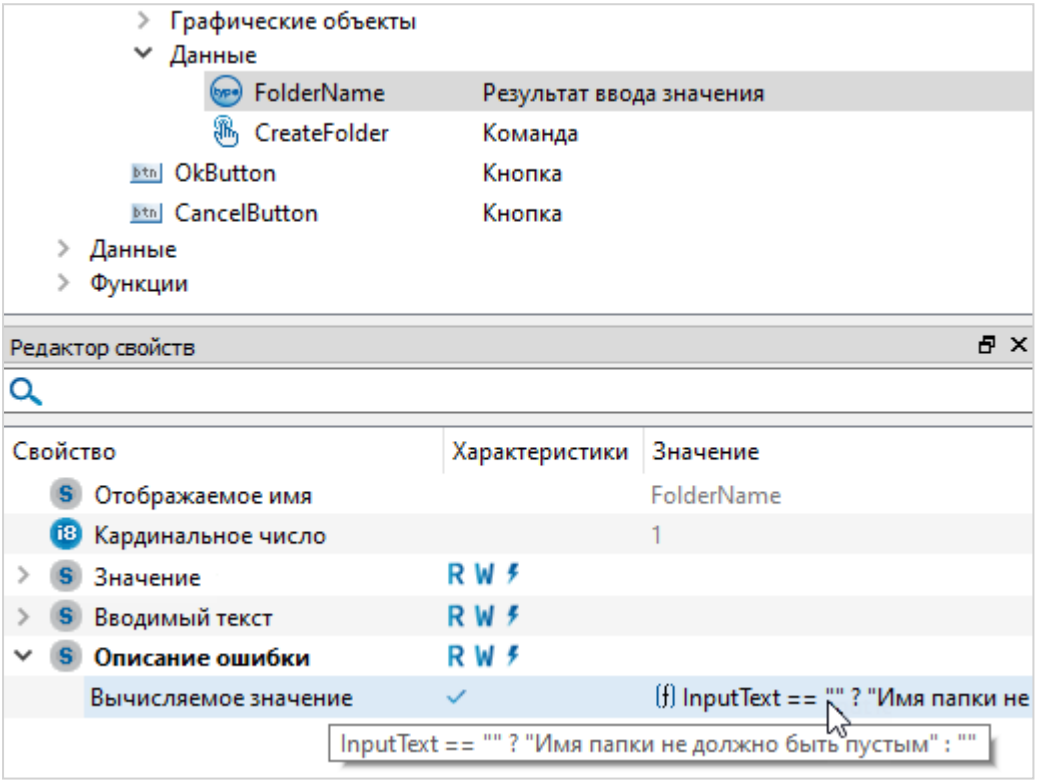
- вокруг поля ввода появляется красная рамка;
- во всплывающей подсказке появляется текст ошибки;
- кнопка **OK** блокируется.

ПРИМЕР

Допустим, значение в поле ввода не должно быть пустым.

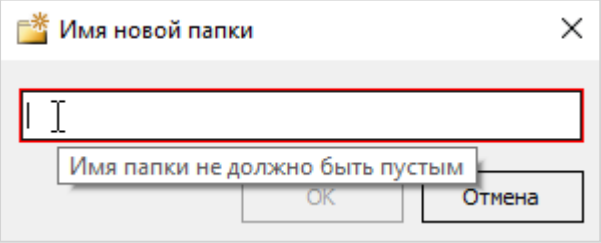
Проверка условия может быть описана на языке Alpha.Om компактной тернарной операцией вида:

```
<condition>?<statement1>:<statement2>
```




В данном случае проверяется вводимый текст (InputText). При пустом значении ввода:

- вокруг поля ввода появится красная рамка;
- во всплывающей подсказке появится текст ошибки;
- кнопка **OK** заблокируется.




## 2.1.15.7. Результат редактирования колонтитула

Компонент, в который сохраняется результат редактирования колонтитула в окне 2.2.8. Редактор колонтитула (стр. 120) - текст и выравнивание. Чтобы использовать результат, обратитесь к свойствам компонента.

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

## Свойства

### Text

Текст колонтитула. Тип значения - уведомляющий string.

### TextAlignment

Выравнивание текста колонтитула. Тип значения - уведомляющий int4.

## 2.1.15.8. Результат выбора файла

Компонент, в котором сохраняется имя и тип файла, выбранного в окне 2.2.9.1. Диалог открытия файла (стр. 122) или 2.2.9.2. Диалог сохранения файла (стр. 127).

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

## Свойства

### Путь к папке (FolderPath)

Хранит полный путь к папке, в которой находится файл, выбранный в окне 2.2.9.1. Диалог открытия файла (стр. 122) или 2.2.9.2. Диалог сохранения файла (стр. 127). Тип значения - уведомляющий string.

### Имя файла (FileName)

Хранит имя файла, выбранного в окне 2.2.9.1. Диалог открытия файла (стр. 122) или 2.2.9.2. Диалог сохранения файла (стр. 127). Тип значения - уведомляющий string.

### Путь к файлу (FilePath)

Хранит полный путь к файлу, выбранному в окне 2.2.9.1. Диалог открытия файла (стр. 122) или 2.2.9.2. Диалог сохранения файла (стр. 127). Тип значения - уведомляющий string.

### Тип файла (FileType)

Хранит индекс типа файла, выбранного в окне 2.2.9.1. Диалог открытия файла (стр. 122) или 2.2.9.2. Диалог сохранения файла (стр. 127). Тип значения - уведомляющий int4.

Индексом типа файла является порядковый номер типа в списке фильтра типа файлов. Список фильтра создается в соответствующем свойстве - **FileTypeFilter (Фильтр типа файлов)** - компонентов Диалог открытия файла (стр. 123) или Диалог сохранения файла (стр. 128).

## 2.1.15.9. Результат диалога авторизации

Компонент, через который передаются id и отображаемое имя текущего пользователя от формы 2.2.10.1. Диалог авторизации пользователя (стр. 133) к форме 2.2.10.2. Диалог смены пароля (стр. 136). Пример

применения приведен в описании компонента 2.2.10.1. Диалог авторизации пользователя (стр. 133).

Экземпляр этого типа не отображается на форме и виден только в области **Структура объекта**.

## Свойства

### UserId

Хранит уникальный идентификатор (uid) текущего пользователя подсистемы безопасности Alpha.Security. Тип значения - уведомляющий string.

ПРИМЕР

Вид значения: alpha:xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx.

Пример значения: **«alpha:c3a997b1-326e-42ee-8907-e359fc17feb7»**.

### UserDisplayName

Хранит отображаемое имя текущего пользователя подсистемы безопасности Alpha.Security. Тип значения - уведомляющий string.

ПРИМЕР

Пример значения: **«Иванов»**.

## 2.1.15.10. Список файлов

Список файлов и папок, содержащихся в указанной папке.

| Имя | Дата изменения | Размер |
|---|---|---|
| configuration_files | 17.11.2022 14:12:29 | |
| new_backups | 17.11.2022 14:12:29 | |
| old_backups | 17.11.2022 14:12:29 | |
| 17112022_bckp.csv | 17.11.2022 14:12:53 | 6377 |
| 17112022_bckp.pdf | 17.11.2022 14:12:53 | 6377 |
| 18112022_bckp.csv | 17.11.2022 14:12:53 | 6377 |
| 18112022_bckp.pdf | 17.11.2022 14:12:53 | 6377 |

Указать папку, содержимое которой будет отображаться в компоненте, настроить фильтры отображения файлов и настроить ограничения на просмотр файловой системы можно с помощью свойств компонента.

## Свойства

### Корневой путь (RootPath)

Ограничивает область просмотра файловой системы. Тип значения - уведомляющий string.

## Отображаемая папка (BrowsedPath)

Путь к папке, содержимое которой должно отображаться в окне 2.1.15.10. Список файлов (стр. 80). Тип значения - уведомляющий string.

Полнота отображения - только папки, папки и все файлы или папки и файлы определенного типа - указывается в свойстве **ExtensionFilter (Фильтр по типу файла)**.

## Фильтр по типу файла (ExtensionFilter)

Фильтр типа файлов, отображаемых в окне 2.1.15.10. Список файлов (стр. 80). Тип значения - уведомляющий string.

- Если не указывать значение, то в списке будут отображаться все файлы и папки.
- Если указать конкретное расширение, например **«`*.csv`»**, то будут отображаться папки и файлы с указанным расширением.

## Показывать свойства файлов (ShowFileProperties)

Позволяет отображать больше информации о файлах и папках в окне 2.1.15.10. Список файлов (стр. 80). Тип значения - уведомляющий bool.

Значение:

- **«`true`»** - для файлов и папок отображаются имена и даты изменений, а для файлов - еще размеры (в байтах);

| Имя | Дата изменения | Размер |
|---|---|---|
| new_backups | 17.11.2022 14:12:29 | |
| old_backups | 17.11.2022 14:12:29 | |
| 17112022_bckp.csv | 17.11.2022 14:12:53 | 6377 |
| 18112022_bckp.csv | 17.11.2022 14:12:53 | 6377 |
| 19112022_bckp.csv | 17.11.2022 14:12:53 | 6377 |

- **«`false`»** - отображаются только имена файлов и папок.

| |
|---|
| new_backups |
| old_backups |
| 17112022_bckp.csv |
| 18112022_bckp.csv |
| 19112022_bckp.csv |

## Разрешена смена папки (ChangeFolderAllowed)

Указывает на возможность смены папки, открывающейся в окне 2.1.15.10. Список файлов (стр. 80). Тип значения - уведомляющий bool.

Значение:

- **«`true`»** - можно сменить открывшуюся папку;
- **«`false`»** - нельзя сменить открывшуюся папку.

## Выделенный путь (SelectedPath)

Путь к файлу или папке, выбранной в окне 2.1.15.10. Список файлов (стр. 80). Тип значения - уведомляющий string.

ПРИМЕР

Допустим, в свойстве **BrowsedPath (Отображаемая папка)** указано значение `D:\CommonLib\backups`, тогда в окне **`FileList`** отображается содержимое этой папки.

| Имя | Дата изменения | Размер |
|---|---|---|
| configuration_files | 17.11.2022 14:12:29 | |
| new_backups | 17.11.2022 14:12:29 | |
| old_backups | 17.11.2022 14:12:29 | |
| 17112022_bckp.csv | 17.11.2022 14:12:53 | 6377 |
| 18112022_bckp.csv | 17.11.2022 14:12:53 | 6377 |

В таком случае свойство **SelectedPath (Выделенный путь)**:

- при выборе папки `new_backups` примет значение **«`D:\CommonLib\backups\new_backups`»**;
- при выборе файла `17112022_bckp.csv` примет значение **«`D:\CommonLib\backups\17112022_bckp.csv`»**.

## Выделенный файл (SelectedFileName)

Название файла, выбранного в окне 2.1.15.10. Список файлов (стр. 80). Тип значения - уведомляющий string.

ПРИМЕР

Допустим, в свойстве **BrowsedPath (Отображаемая папка)** указано значение `D:\CommonLib\backups`, тогда в окне **`FileList`** отображается содержимое этой папки.

| Имя | Дата изменения | Размер |
|---|---|---|
| configuration_files | 17.11.2022 14:12:29 | |
| new_backups | 17.11.2022 14:12:29 | |
| old_backups | 17.11.2022 14:12:29 | |
| 17112022_bckp.csv | 17.11.2022 14:12:53 | 6377 |
| 18112022_bckp.csv | 17.11.2022 14:12:53 | 6377 |

В таком случае:

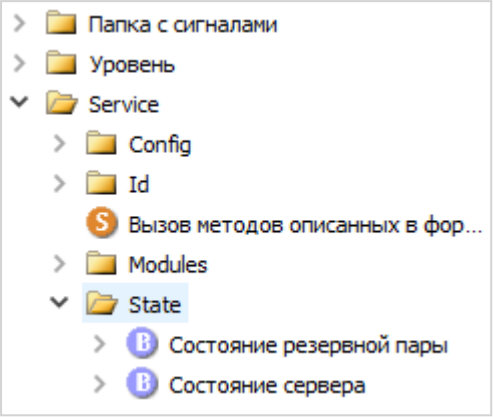
- при выборе файла `17112022_bckp.csv` свойство **SelectedFileName (Выделенный файл)** примет значение **«`17112022_bckp.csv`»**;
- при выборе папки:
  - если ранее не был выбран ни один файл, то значение свойства **SelectedFileName (Выделенный файл)** останется пустым;
  - если ранее был выбран какой-либо файл, то имя этого файла будет храниться в качестве значения свойства **SelectedFileName (Выделенный файл)**, пока не будет выбран другой файл и значение не перезапишется.

## OnPathChosen

Ссылка на команду, выполняемую в результате двойного щелчка ЛКМ по строке файла в списке.

## 2.1.16. Дерево сигналов

Дерево сигналов. Предназначено для отображения адресного пространства сервера в виде дерева.




### 2.1.16.1. Свойства

#### Вывод описания предпочтительнее имени (DescriptionPreferred)

Позволяет выбрать, что отображать в дереве: имя сигнала или его описание. Тип значения - уведомляющий bool.

Значение:

- «`true`» - отображать описание сигнала (если не указано, то имя);
- «`false`» - отображать имя сигнала.

#### pMouseRightClicked

Ссылка на команду, выполняемую при клике правой кнопкой мыши по узлу дерева.

Может использоваться, например, для вызова контекстного меню (стр. 107).

#### Показывать столбец атрибутов (ShowAttributesColumn)

Отображает столбец с иконками атрибутов сигнала справа от названия каждого сигнала в дереве. По иконкам можно понять, является ли сигнал источником событий, и ведется ли для него запись истории значений. Тип значения - уведомляющий bool.

Значение:

- «`true`» - отображать столбец атрибутов;
- «`false`» - не отображать столбец атрибутов.

При выборе значения «`true`» будут отображаться иконки атрибутов:

- - ведется запись истории значений сигнала;
- - сигнал является источником события.

## Ссылка на источник (init_ApSource)

Здесь указывается сервер, дерево сигналов которого должно отображаться в **ItemTree**. Тип значения - ссылка на источник данных типа **Источник АР**.

## Фильтр (init_Filter)

Позволяет настроить отображение в дереве сигналов, для которых ведется запись истории значений, а также те из них, которые являются источником события.

Чтобы настроить фильтрацию:

1. В контекстном меню свойства выберите **Указать объект**. Если свойство **Фильтр** автоматически не развернулось, то разверните его.
2. Для свойства фильтра **Включен** выберите **«true»**.
3. Для свойства фильтра **С историей значений** или **Источники событий** выберите **«true»**, если в дереве необходимо отобразить такие сигналы. Если такие сигналы необходимо скрыть, то для соответствующего свойства фильтра выберите **«false»**.

ПРИМЕЧАНИЕ

Если для какого-либо узла все дочерние сигналы скрыты после фильтрации, то такой узел будет отображаться в дереве.

## pDragStarted

При перетаскивании узла дерева левой кнопкой мыши выполняет код экземпляра типа 2.1.8. Обработчик события (стр. 36), ссылка на который указана в свойстве. Также передает информацию об узле.

Информация об узле представлена в виде значений следующих атрибутов:

- `tag` - адрес сигнала;
- `dataType` - тип данных;
- `isWritable` - возможность изменения сигнала пользователем;
- `isHistorized` - запись истории значений сигнала;
- `isEventSource` - сигнал является источником события;
- `icon` - путь к файлу иконки.

Атрибуты можно вызвать в коде обработчика события.

ПРИМЕР

Пример использования атрибута сигнала в коде обработчика события:

```
var item_info = SignalTree.GetCurrentItemInfo();
var info = JSON.parse(item_info);
if(info.isWritable == true)
{
 LogView.AddMessage(1,"Новое значение может быть записано");
}
else
{
 LogView.AddMessage(3,"Невозможно записать новое значение");
}
```

В переменную `item_info` сохраняется набор значений атрибутов, предоставляемый функцией **GetCurrentItemInfo()**. В 2.1.17. Журнал сообщений (стр. 88) выводится сообщение о возможности изменения значения выбранного сигнала.

## pDoubleClicked

При двойном нажатии на узле дерева левой кнопкой мыши выполняет код экземпляра типа 2.1.8. Обработчик события (стр. 36), ссылка на который указана в свойстве. Также передает информацию об узле.

Информация об узле представлена в виде значений следующих атрибутов:

- `tag` - адрес сигнала;
- `dataType` - тип данных;
- `isWritable` - возможность изменения сигнала пользователем;
- `isHistorized` - запись истории значений сигнала;
- `isEventSource` - сигнал является источником события;
- `icon` - путь к файлу иконки.

Атрибуты можно вызвать в коде обработчика события.

ПРИМЕР

Пример использования атрибута сигнала в коде обработчика события:

```
var item_info = SignalTree.GetCurrentItemInfo();
var info = JSON.parse(item_info);
if(info.isWritable == true)
{
 LogView.AddMessage(1,"Новое значение может быть записано");
}
else
{
 LogView.AddMessage(3,"Невозможно записать новое значение");
}
```

В переменную `item_info` сохраняется набор значений атрибутов, предоставляемый функцией **GetCurrentItemInfo()**. В 2.1.17. Журнал сообщений (стр. 88) выводится сообщение о возможности изменения значения выбранного сигнала.

## 2.1.16.2. Функции

### Find

```
void Find(string pattern)
```

Выполняет поиск сигналов в дереве по шаблону названия, указанному в качестве значения входного параметра pattern.

Поиск выполняется на стороне сервера. Результат поиска отображается в дереве.

### CancelFind

```
void CancelFind()
```

Отменяет поиск сигналов, выполняемый функцией **Find()**.

После выполнения этой функции в дереве сигналов вместо результатов поиска отображаются все сигналы сервера.

Функция не требует входных параметров.

### RefreshCurrentItem

```
void RefreshCurrentItem()
```

Заново запрашивает список дочерних узлов для узла, выделенного в дереве.

Функция не требует входных параметров.

## GetCurrentItemInfo

```
string GetCurrentItemInfo()
```

Возвращает описание выделенного в дереве сигнала или папки в виде json-строки.

Функция не требует входных параметров.

ПРИМЕР

Пример возвращаемого описания для выделенного сигнала:

```
{
        "tag": "Level.s_level",
        "dataType": "int1",
        "isWritable": "true",
        "isHistorized": "true",
        "isEventSource": "true",
        "icon":"CommonLib_Icons/Items/SByte.png"
}
```

ПРИМЕЧАНИЕ

Возвращаемое описание для выделенной папки не содержит атрибут `dataType`.

ПРИМЕР

Пример вызова функции в обработчике команды, выполняемой в момент начала перетаскивания узла:

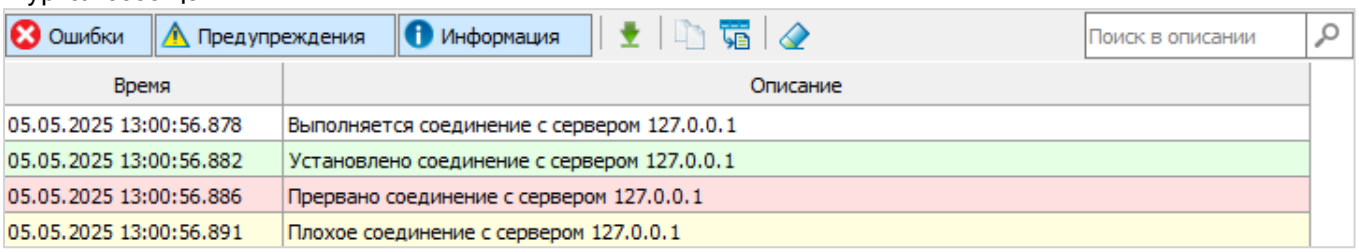
```
// получение значения, возвращаемого функцией
var json = GetCurrentItemInfo();

// разбор полученного объекта json
if (json != "")
{
 var info = JSON.parse(json);

 Drag.Image = info.icon;
 Drag.SetText("{\"data\":[" + json + "]}");
 Drag.DoDragDrop();
}
```

## 2.1.17. Журнал сообщений

Журнал сообщений.




Отображает сообщения в виде таблицы. По умолчанию в таблице отображаются только **Время** и **Описание** сообщений. Чтобы выводить дополнительную информацию, измените значение свойства AdditionalColumns (стр. 89).

Сообщения в журнале могут иметь разную важность, обозначаемую цветом фона сообщения:

- ошибки - красные;
- предупреждения - желтые;
- информационные сообщения: обычные - зеленые, и отладочные - белые.

Подсказки по работе с журналом, добавленным в проект, в режиме исполнения:

- чтобы в журнале не отображались сообщения определенного типа, отожмите кнопку с названием типа на панели инструментов журнала;
- чтобы следить за новыми сообщениями, нажмите кнопку **Автоматическое прокручивание списка** на панели инструментов;
- чтобы скопировать выделенные сообщения в буфер обмена, нажмите кнопку **Копировать выделенные сообщения** на панели инструментов или выберите такой пункт в контекстном меню журнала;
- чтобы сохранить журнал сообщений, нажмите кнопку **Экспорт в файл...** на панели инструментов или выберите такой пункт в контекстном меню журнала;
- чтобы очистить журнал сообщений, нажмите кнопку **Очистить список** на панели инструментов или выберите такой пункт в контекстном меню журнала;
- чтобы найти в журнале сообщения с определенной фразой, введите ее в поле **Поиск в описании** и нажмите кнопку **Поиск** ;
- чтобы скопировать описание выделенных сообщений в буфер обмена, выберите **Копировать описание** в контекстном меню журнала или используйте "горячие" клавиши «Ctrl»+«C».

### 2.1.17.1. Свойства

#### IsVisible

Меняет видимость журнала. Тип значения - уведомляющий bool.

Значение:

- «true» - показать журнал;
- «false» - скрыть журнал.

Еще изменить видимость журнала можно при помощи 2.1.14.2. Кнопка-индикатор журнала сообщений (стр. 71).

## Папка для экспорта (ExportPath)

Путь к папке, в которой по умолчанию сохраняются файлы. Тип значения - string.

## Доступность файловой системы (FileSystemAccess)

Ограничивает смену и создание папки в диалоге сохранения файла. Тип значения - bool.

Значение:

- **«true»** - разрешить доступ к файловой системе;
- **«false»** - запретить доступ к файловой системе.

При выборе значения **«false»** файлы сохраняются в папку, указанную в свойстве **Папка для экспорта**.

## Количество ошибок (ErrorCount)

Хранит текущее количество сообщений об ошибках в журнале. Тип значения - уведомляющий int4.

## Количество предупреждений (WarningCount)

Хранит текущее количество предупреждений в журнале. Тип значения - уведомляющий int4.

## Количество сообщений (InformationCount)

Хранит текущее количество информационных сообщений в журнале. Тип значения - уведомляющий int4.

## Дополнительные столбцы (AdditionalColumns)

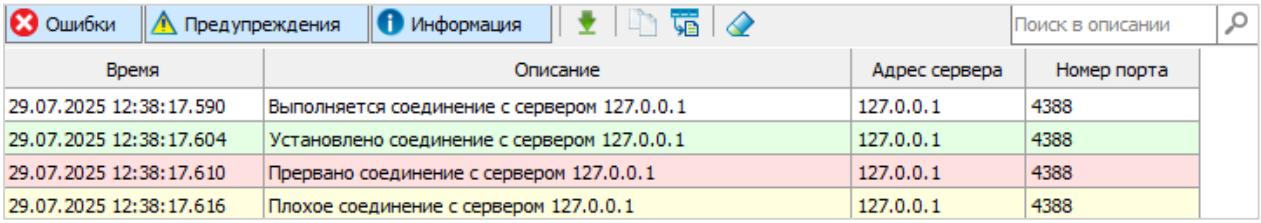
Перечень дополнительных столбцов таблицы сообщений в виде json-строки. Тип значения - string.

ПРИМЕР

Пример значения свойства:

```
[
{"id":"address", "title":"Адрес сервера"},
{"id":"port", "title":"Номер порта"}
]
```

Результат в рантайме - новые столбцы **Адрес сервера** и **Номер порта**:




| Время | Описание | Адрес сервера | Номер порта |
|---|---|---|---|
| 29.07.2025 12:38:17.590 | Выполняется соединение с сервером 127.0.0.1 | 127.0.0.1 | 4388 |
| 29.07.2025 12:38:17.604 | Установлено соединение с сервером 127.0.0.1 | 127.0.0.1 | 4388 |
| 29.07.2025 12:38:17.610 | Прервано соединение с сервером 127.0.0.1 | 127.0.0.1 | 4388 |
| 29.07.2025 12:38:17.616 | Плохое соединение с сервером 127.0.0.1 | 127.0.0.1 | 4388 |

### Максимальное количество строк (MaxRowCount)

Максимальное количество строк сообщений, отображаемых в журнале. Тип значения - int4.

При превышении указанного здесь количества строк устаревшие сообщения будут автоматически удалены.

### Внешний вид изменен (pViewChanged)

Ссылка на экземпляр типа **Команда**, который активируется при:

- изменении ширины столбцов;
- включении и отключении отображения какого-либо типа сообщения;
- включении и отключении автоматического прокручивания списка сообщений;
- выборе пути и типа файла в диалоге экспорта в файл;
- поиске в описании сообщения.

Получить измененные настройки внешнего вида журнала сообщений можно в обработчике события Invoked экземпляра типа **Команда**. Для этого следует использовать функцию GetViewSettings() экземпляра типа **Журнал сообщений**. Подробнее об этой функции написано в разделе 2.1.17.2. Функции (стр. 90).

## 2.1.17.2. Функции

### AddMessage

```
void AddMessage(int4 importance, string text)
```

Добавляет в журнал сообщение с заданной важностью. Если в свойстве AdditionalColumns (стр. 89) указаны дополнительные столбцы, используйте функцию Add().
Входные параметры:

- важность;
- текст сообщения.

Важность обозначается цифрой:

- **«0»** - отладочное информационное сообщение, цвет фона - белый;
- **«1»** - информационное сообщение, цвет фона - зеленый;
- **«2»** - важное сообщение, цвет фона - желтый;
- **«3»** - сообщение об ошибке, цвет фона - красный.

ПРИМЕР

```
LogView.AddMessage(3, "Прервано соединение с сервером 127.0.0.1")
```

### Add

```
void Add(int4 importance, string text, string additional)
```

Добавляет в журнал сообщение с заданной важностью и дополнительными данными. Эта функция используется, если в свойстве AdditionalColumns (стр. 89) указаны дополнительные столбцы.

Входные параметры:

- важность;
- текст сообщения;
- дополнительные данные в виде json-строки:
  { "id1":value1, "id2":value2, ... },
  помещаемые в дополнительные столбцы таблицы сообщений.

ПРИМЕР

При вызове функции в JavaScript понадобится метод JSON.stringify() для преобразования данных в json-строку:

```
Log.Add(3, "Прервано соединение с сервером 127.0.0.1", JSON.stringify
({"address":"127.0.0.1", "port":4388}))
```

При вызове функции в Alpha.Om представьте данные в виде строки, заключив их в кавычки, а кавычки внутри строки экранируйте обратным слешем:

```
Log.Add(3, "Прервано соединение с сервером 127.0.0.1", "{
\"address\":\"127.0.0.1\", \"port\":4388 }")
```

## Clear

```
void Clear()
```

Очищает журнал сообщений.

## GetViewSettings

```
string GetViewSettings()
```

Возвращает настройки отображения журнала в виде json-строки.

Настройки отображения содержат информацию о ширине столбцов, порядке сортировки, включенных кнопках фильтрации, текст поиска, режим слежения за последним сообщением, путь к папке для экспорта, расширение экспортируемых файлов. Эти настройки могут использоваться для восстановления отображения журнала с помощью функции UpdateView() при перезапуске проекта.

ПРИМЕР

Пример возвращаемого значения:

```
{
"Columns": {
"importance": {
"Index": 0,
"Width": 20,
"Hidden": true
},
"time": {
"Index": 1,
"Width": 150,
"Hidden": false
},
"text": {
"Index": 2,
"Width": 424,
"Hidden": false
}
},
"Filter":
{
"Errors": true,
"Warnings": true,
"Informations": true,
"Text": ""
},
"FollowLatest": false,
"ExportToFile":
{
"Folder":"",
"FileType":0
}
}
```

## UpdateView

```
void UpdateView(string settings)
```

Настраивает отображение журнала в соответствии с настройками, которые могут быть получены в результате вызова функции GetViewSettings().

Входной параметр - настройки отображения журнала в виде json-строки.

## 2.1.17.3. Команды

### CopyDescription

Копирует в буфер обмена описание выделенных сообщений.

ПРИМЕР

```
LogView.CopyDescription.Invoke();
```

### CopySelection

Копирует в буфер обмена текст выделенных ячеек таблицы, разделенных символом табуляции.

ПРИМЕР

```
LogView.CopySelection.Invoke();
```

### ExportToFile

Открывает диалог сохранения журнала сообщений в файл.

ПРИМЕР

```
LogView.ExportToFile.Invoke();
```

### ClearList

Очищает журнал сообщений.

ПРИМЕР

```
LogView.ClearList.Invoke();
```

## 2.1.18. Редактируемая таблица

Таблица с возможностью редактирования значений ячеек.

| Тип данных | AllTypes | ValuesColumn |
|---|---|---|
| int1 | 2 | |
| int2 | 22 | |
| int4 | 222 | |
| int8 | 2222 | |
| uint1 | 0 | [0, 1, 2, 3, 4, 5, ... |
| uint2 | | |
| uint4 | | |
| uint8 | | |
| float | | |
| double | | |
| bool | [x] | [0,1] |
| string | stringText | |

В экземпляре типа **Редактируемая таблица** можно:

- конфигурировать столбцы таблицы на основе модели данных или с помощью функции;
- использовать тему оформления для декорирования таблицы с возможностью переопределить декоратор;
- добавлять, удалять и перемещать строки с помощью клавиш;
- проверять корректность ввода в редакторе значений;
- добавлять пользовательские редакторы значений с помощью типа 2.1.19. Редактор ячейки таблицы (стр. 105);
- использовать встроенные типы редакторов значений.

Тип является наследником типа **Таблица (Table)**, предоставляемого расширением Alpha.HMI.Tables, поэтому имеет похожий принцип использования в проекте и такие же свойства, функции, события. Чтобы начать работу с таблицей, необходимо задать модель данных с помощью экземпляра типа **Модель данных**, предоставляемого расширением Alpha.HMI.Tables. Подробнее об этом - в документации на Alpha.HMI (содержит документацию для Alpha.HMI.Tables), кратко - ниже.

## Пример использования в проекте

### Заполнение таблицы на основе модели данных

1. Поместите в рабочую область экземпляры типов **Таблица: модель данных** и **Редактируемая таблица**.
2. Создайте модель данных. Сделать это можно либо вручную, либо заполнив модель данными от источника. Подробнее об этом - в документации на Alpha.HMI, раздел "Формирование таблиц".
3. Сошлитесь на созданный экземпляр **Таблица: модель данных** из свойства Модель данных вашего экземпляра типа **Редактируемая таблица**.
4. Вызовите функцию ConfigureByModel() для вашего экземпляра типа **Редактируемая таблица**.

ПРИМЕР

Пусть нужно создать таблицу с двумя колонками. В одной можно будет указать должность сотрудника, в другой - фамилию. Значение в обеих колонках вводится вручную.

Для создания таблицы с двумя колонками и тремя строками можно использовать следующий код, написанный на языке Alpha.Om:

```
TableModel.AddColumn("title", "","","","string", false, true); /* функция
добавляет
в модель данных столбец для указания должности с идентификатором title,
без заголовка, без описания, с типом данных string,
с возможностью записи значения, с разрешением не содержать данных в
ячейках */
TableModel.AddColumn("surname", "","","","string", false, true); /* такой
же столбец,
но для ввода фамилии */
EditableTable.ConfigureByModel(); // таблица конфигурируется в
соответствии с моделью данных
for (i: int4 = 0; i <= 2; i += 1)
 EditableTable.AddRow(); // таблица заполняется тремя строками
```

Полученная таблица будет выглядеть так:

| title | surname |
|---|---|
| | |
| | |
| | |

## Переопределение таблицы с изменением типа редактора ячейки

Столбцы таблицы, сформированной с помощью функции ConfigureByModel(), можно переопределить. Для этого вызовите функцию Configure() для вашего экземпляра типа **Редактируемая таблица**, указав в качестве входного значения JSON-строку, описывающую переопределенную модель таблицы.

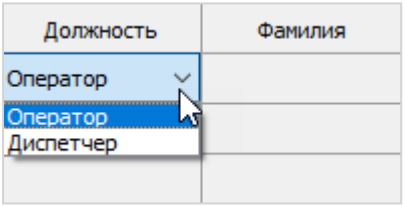
ПРИМЕР

Пусть в таблице, созданной ранее, нужно указать заголовки столбцов, а редактор ячейки одного из столбцов заменить на выпадающий список с возможными значениями.

Для этого можно использовать следующий код, написанный на языке JavaScript:

```
var config = { // создается объект
 'columns': [
  { 'id': "title", 'title': "Должность", 'values': [ "Оператор",
"Диспетчер" ], 'editorType': "1" },
  /* для первого столбца указывается заголовок, тип редактора заменен на
выпадающий список (1)
  с возможными значениями "Оператор" и "Диспетчер" */
  { 'id': "surname", 'title': "Фамилия" } // для второго столбца указан
заголовок
 ]
}
Configure(JSON.stringify(config)); /* таблица переопределяется в
соответствии с JSON-строкой,
полученной на основе описанного объекта */
```

Переопределенная таблица будет выглядеть так:




В режиме рантайма можно:

- Добавить строку, нажав клавишу Insert.
- Удалить строку, нажав сочетание клавиш Ctrl+Delete.

## 2.1.18.1. Свойства

### Модель данных (TableModel)

Ссылка на экземпляр типа **`Таблица: модель данных`**, предоставляемый Alpha.HMI.Tables.

Таблица: модель данных формирует невизуальную модель, предназначенную для хранения данных в табличном виде.

### Возможность добавления строк (CanAddRows)

Разрешает добавление новых строк в таблицу. Тип значения - уведомляющий bool.

Значение:

- **«`true`»** - добавление новых строк разрешено;
- **«`false`»** - добавление новых строк запрещено.

ПРИМЕЧАНИЕ
Чтобы добавить новую строку, используйте клавишу «**Insert**».

## Возможность удаления строк (CanDeleteRows)

Разрешает удаление строк из таблицы. Тип значения - уведомляющий bool.

Значение:

- «**true**» - удаление строк разрешено;
- «**false**» - удаление строк запрещено.

ПРИМЕЧАНИЕ
Чтобы удалить строку, используйте сочетание клавиш «**Ctrl**»+«**Delete**».

## Возможность перемещения строк (CanMoveRows)

Разрешает перемещение строк в таблице. Тип значения - уведомляющий bool.

Значение:

- «**true**» - перемещение строк разрешено;
- «**false**» - перемещение строк запрещено.

ПРИМЕЧАНИЕ
Чтобы переместить строку, используйте сочетание клавиш:

- «**Ctrl**»+«↑» для перемещения выше;
- «**Ctrl**»+«↓» для перемещения ниже.

## В режиме редактирования (InEditMode)

Предоставляет информацию о состоянии редактирования значения ячейки. Тип значения - уведомляющий bool.

Только для чтения в режиме рантайма.

Значение:

- «**true**» - в данный момент редактируется значение какой-либо из ячеек;
- «**false**» - в данный момент редактирование значений ячеек не выполняется.

## Текущая строка (CurrentRow)

Предоставляет номер строки выделенной ячейки. Тип значения - уведомляющий int4.

Только для чтения в режиме рантайма.

## Текущий столбец (CurrentColumn)

Предоставляет номер столбца выделенной ячейки. Тип значения - уведомляющий int4.

Только для чтения в режиме рантайма.

## Декорировать таблицу (pSetDecorator)

Ссылка на команду, определяющую настройки декоратора таблицы.

По умолчанию указана ссылка на команду, декорирующую таблицу.

## Событие нажатия клавиши (pKeyPress)

Ссылка на 2.1.8. Обработчик события (стр. 36), выполняемый при нажатии клавиши в редактируемой таблице.

По умолчанию указана ссылка на обработчик события, выполняющий:

- если в данный момент редактируется значение ячейки, а значит свойство В режиме редактирования (InEditMode) принимает значение «**true**»:
  - по нажатию клавиши «**Enter**» - завершение редактирования ячейки с сохранением значения;
  - по нажатию клавиши «**Esc**» - завершение редактирования ячейки без сохранения значения.
- если в данный момент не редактируется значение ячейки, а значит свойство В режиме редактирования (InEditMode) принимает значение «**false**»:
  - по нажатию клавиши «**F2**» или любого символа - переход в режим редактирования ячейки;
  - по нажатию клавиши «**Insert**» при условии, что свойство Возможность добавления строк принимает значение «**true**» - добавление новой строки;
  - по нажатию клавиши «**Delete**» при условии, что свойство Поведение выделения принимает значение «**Выделять строки**» - удаление выделенных строк;
  - по нажатию клавиши «**Delete**» при условии, что свойство Поведение выделения принимает значение «**Выделять элементы**» - очистку содержимого ячеек;
  - по нажатию клавиш «**Ctrl**»+«**Delete**» при условии, что свойство Поведение выделения принимает значение «**Выделять элементы**» - удаление строк, содержащих выделенные ячейки.

Если переопределить ссылку на обработчик события, указанный по умолчанию, то в таком обработчике можно использовать аргументы возникшего события. Эти аргументы хранятся в виде JSON-строки в Буфере аргументов события (стр. 34). Получить их можно в обработчике события с помощью функции Args.Get().

ПРИМЕР

Пример JSON-строки, содержащей аргументы события, возникшего по нажатию клавиши **«5»** в редактируемой таблице

```
{
 "keyCode": 5,
 "modifiers": 0
}
```

Здесь:

- `keyCode` - код нажатой клавиши. Полный список кодов клавиш приведён в Справочном руководстве на Alpha.HMI → Компоненты окружения → Оборудование: клавиатура → Функции → подраздел Коды клавиш.
- `modifiers` - код клавиш-модификаторов (например, **«Ctrl»**, **«Shift»**, **«Alt»**), которые были зажаты в момент события. Если нажаты несколько таких клавиш, их значения суммируются. Полный список кодов модификаторов приведён в Справочном руководстве на Alpha.HMI → Компоненты окружения → Оборудование: клавиатура → Функции → подраздел Комбинации модификаторов.

## Вставить новую строку (pInsertNewRow)

Ссылка на команду, выполняемую при попытке вставить новую строку в таблицу.

По умолчанию указана ссылка на команду, вставляющую строку в таблицу.

Если переопределить ссылку команду, указанную по умолчанию, то при попытке вставить новую строку будет выполнена новая команда, а строка добавлена не будет. Чтобы добавить строку программно, используйте функцию InsertRowAfterSelection() (стр. 104).

## Удалить выделенные строки (pRemoveSelectedRows)

Ссылка на команду, выполняемую при попытке удалить выбранную строку из таблицы.

По умолчанию указана ссылка на команду, удаляющую выбранную строку из таблицы.

Если переопределить ссылку команду, указанную по умолчанию, то при попытке удалить выбранную строку будет выполнена новая команда, а строка удалена не будет. Чтобы удалить строку программно, используйте функцию RemoveRows() (стр. 104).

## Переместить выше (pUpSelectedRows)

Ссылка на **Пользовательскую команду**, активируемую при перемещении строки выше. Чтобы использовать это свойство, установите значение **«true»** для свойства Возможность перемещения строк (CanMoveRows).

По умолчанию указана ссылка на пользовательскую команду, выполняющую перемещение строки на одну выше.

Если переопределить ссылку, указанную по умолчанию, то при попытке переместить строку выше будет выполнена новая команда, а перемещение строки выше выполнено не будет.

## Переместить ниже (pDownSelectedRows)

Ссылка на **Пользовательскую команду**, активируемую при перемещении строки ниже. Чтобы использовать это свойство, установите значение **«true»** для свойства Возможность перемещения строк (CanMoveRows).

По умолчанию указана ссылка на пользовательскую команду, выполняющую перемещение строки на одну ниже.

Если переопределить ссылку, указанную по умолчанию, то при попытке переместить строку ниже будет выполнена новая команда, а перемещение строки ниже выполнено не будет.

## Копировать (pCopyToClipboard)

Ссылка на **Пользовательскую команду**, которая помещает в буфер обмена текст выделенных ячеек таблицы, разделенных символом табуляции.

По умолчанию указана ссылка на пользовательскую команду, которая помещает в буфер обмена текст выделенных ячеек таблицы, разделенных символом табуляции.

Если переопределить ссылку, указанную по умолчанию, то при попытке скопировать текст выделенных ячеек таблицы будет выполнена новая команда, а в буфер обмена не будет помещен текст этих ячеек.

## Начать редактирование (pStartEdit)

Ссылка на 2.1.8. Обработчик события (стр. 36), активируемый в момент начала редактирования ячейки.

По умолчанию указана ссылка на обработчик события, выполняющий проверку возможности редактирования ячейки, а затем - редактирование текстового значения.

Если переопределить ссылку, указанную по умолчанию, то при попытке редактировать значение будет выполнена новая команда, а проверка возможности редактирования и само редактирование значения выполнены не будут. В указанный обработчик в виде json-объекта будут переданы следующие аргументы:

```
{
 'row': <номер строки>,
 'column': <номер столбца>,
 'value': <текст, введенный пользователем>
}
```

Получить аргументы в обработчике можно с помощью функции **Args.Get()** обработчика событий. Подробнее об этом - в описании типа 2.1.8. Обработчик события (стр. 36).

Переопределение ссылки на обработчик события может понадобиться для изменения вида редактора значения ячейки и перечисления возможных значений в этом редакторе. Для этого используйте функцию StartEditCell() (стр. 104). В качестве входного параметра укажите JSON-строку:

```
{
 'row': <номер строки>,
 'column': <номер столбца>,
 'dataType': <тип данных>,
```

```
 'value': <редактируемое значение>,
 'values': <массив значений для выпадающего списка>,
 'editorType': <тип редактора: 0 - текстовый, 1 - выпадающий список, 2 - флаг, 3
- целого числа, 4 - даты и времени, 5 - даты, 6 - времени>
}
```

Поля `row` и `column` обязательны. Остальные указываются в зависимости от задачи:

- Для изменения вида редактора укажите одно из следующих значений для свойства `editorType`:
  - **«0»** - текстовый редактор;
  - **«1»** - редактор в виде выпадающего списка;
  - **«2»** - редактор в виде флага;
  - **«3»** - редактор целого числа;
  - **«4»** - редактор даты и времени;
  - **«5»** - редактор даты;
  - **«6»** - редактор времени.

  Обратите внимание, что можно создавать собственные редакторы - с помощью типа 2.1.19. Редактор ячейки таблицы (стр. 105). Если в вашем проекте создан собственный редактор, то значение для него указано в его свойстве Тип редактора (EditorType).
- Перечень возможных значений должен быть указан в качестве массива-значения свойства `values`.

## Проверить значение (pValidateValue)

Ссылка на 2.1.8. Обработчик события (стр. 36), проверяющий соответствие значения ячейки типу данных.

## Завершить редактирование (pFinishEdit)

Ссылка на команду, выполняемую при завершении редактирования значения ячейки:

- при потере фокуса у редактора - с сохранением отредактированного значения.
- при нажатии клавиши Enter - с сохранением отредактированного значения.

По умолчанию указана ссылка на команду, присваивающую введенное значение ячейке, если оно указано без ошибок.

Если переопределить ссылку, указанную по умолчанию, то при завершении редактирования будет выполнена новая команда, а проверка и установка значения выполнены не будут. Чтобы присвоить значение ячейке программно, используйте функцию SetCellData() текущего экземпляра типа **Таблица: модель данных**. Пример вызова функции:

```
EditableTable.TableModel.SetCellData(row,column,Editor.Value);
```

## Сделаны изменения (pChanged)

Ссылка на команду, выполняемую при внесении изменений в таблицу.

По умолчанию указана ссылка на команду, не выполняющую никаких действий.

### Редактор ячейки таблицы (pEditor)

Ссылка на экземпляр типа 2.1.19. Редактор ячейки таблицы (стр. 105).

По умолчанию указана ссылка на стандартный тип редактора, который можно менять (текстовый редактор, редактор в виде выпадающего списка, редактор в виде флага, редактор целого числа, редактор даты и времени, редактор даты и редактор времени). Подробнее об этом - в описании свойства-ссылки Начать редактирование (pStartEdit) (стр. 100).

Если переопределить ссылку, указанную по умолчанию, можно будет использовать новый вид редактора для ячеек таблицы. Подробнее об этом - в описании типа 2.1.19. Редактор ячейки таблицы (стр. 105).

### Внешний вид изменен (pViewChanged)

Ссылка на команду, выполняемую при изменении внешнего вида таблицы.

ПРИМЕЧАНИЕ

Под внешним видом на данный момент подразумевается только ширина столбцов.

### Ширина столбца изменена (pColumnWidthChanged)

Ссылка на 2.1.8. Обработчик события (стр. 36), активируемый при изменении ширины столбцов.

По умолчанию указана ссылка на обработчик события, не выполняющий никаких действий.

Если переопределить ссылку, указанную по умолчанию, то при изменении ширины столбца будет активирован указанный экземпляр обработчика события, и в него в виде json-объекта будут переданы следующие аргументы:

```
{
 'index': <идентификатор столбца>,
 'width': <новая ширина столбца>
}
```

Получить аргументы в обработчике можно с помощью функции **Args.Get()** обработчика событий. Подробнее об этом - в описании типа 2.1.8. Обработчик события (стр. 36).

Переопределение ссылки на обработчик события можно использовать для сохранения внешнего вида таблицы.

## 2.1.18.2. Функции

### Configure

```
void Configure(string json)
```

Переопределяет столбцы таблицы, сформированной с помощью функции ConfigureByModel() (стр. 104) на основе модели данных.